

From Bits to Intelligence

从比特到智能

芯片 · 系统 · 算法 · AI

Liuhuachao

liuhuachaohaha@gmail.com

2026 年 6 月 18 日

目录

1	项目介绍	1
1.1	目录结构	1
1.2	阅读顺序建议	2
2	用户态 AArch64 汇编	3
2.1	系统调用：最底层的 I/O	3
2.1.1	为什么需要系统调用	3
2.1.2	exit.s：三个系统调用的封装	3
2.1.3	代码解读	4
2.2	堆内存分配器	4
2.2.1	设计思路	4
2.2.2	heap 区的定义与初始化	5
2.2.3	memory_init：初始化堆	6
2.2.4	memory_alloc：分配一块内存	6
2.2.5	memory_free：释放内存并合并相邻空闲块	8
2.2.6	测试内存分配器	10
2.3	字符串复制：用户态的 strcpy	11
2.3.1	copy_string：从 input_buf 复制到新分配的堆内存	11
2.4	平行数组：字符串的索引存储	12
2.4.1	为什么需要索引存储	12
2.4.2	string_array.s：数据区定义	13
2.4.3	store_string_ptr_len：写入一对指针和长度	13
2.4.4	print_index_strings：按序号把字符串打印出来	14
2.5	命令解析：find_two_spaces	15
2.5.1	把命令切成 token 的第一步	15
2.5.2	find_two_spaces：单遍扫描找两个空格	16
2.6	主程序：把一切串起来	17
2.6.1	hello.s：主程序入口	17
2.6.2	store_string：读入并保存一行	18
2.6.3	get_string_ptr_len_spaces：打印并解析空格	19
2.6.4	理解整体数据流	19
2.6.5	主循环限制	19
2.6.6	新版主程序：把解析后的 token 逐条打印	19
2.6.7	get_instr_0/1/2/3：从指针数组里拿一个 token 打印	20
2.6.8	print_newline：输出一个换行符	21

2.6.9	现在整个程序的工作流	22
2.7	编译与调试脚本	22
2.7.1	clang: 汇编成可执行文件	22
2.7.2	为什么不用 ld 不用自己链接	23
2.7.3	调试流程	23
2.7.4	运行流程	23
2.8	指令解析: 把命令切成 token 指针数组	24
2.8.1	数据区: instr_addr 全局指针数组	24
2.8.2	store_instr_addr: 把一条命令切成 token 并存下	25
2.8.3	get_instr_addr: 按序号取回 token 指针数组	27
2.8.4	为什么要单独复制 token?	28
3	QEMU 裸机 AArch64	29
3.1	start.s: 最简裸机程序	29
3.1.1	极简的裸机程序	29
3.1.2	和 ISA 章节的差异	30
3.2	run.sh: 交叉编译与启动脚本	31
3.2.1	为什么需要三步式构建	31
3.2.2	常见错误	32
3.3	理解地址空间与 UART	33
3.3.1	裸机地址空间的样子	33
3.3.2	PL011 UART 的寄存器表 (关键部分)	33
3.3.3	如果要从用户键盘接收数据	34
3.3.4	内存与硬件设备的区别	34
4	Verilog 芯片设计	35
4.1	Verilog 基本概念	35
4.1.1	软件和硬件的根本区别	35
4.1.2	同步逻辑: 时钟驱动一切	35
4.1.3	UART 通信的基本约定	36
4.2	adder.v: 64 位字节加法器	36
4.2.1	设计目标	36
4.2.2	源码与讲解	36
4.2.3	一个更简洁的写法	37
4.3	uart_tx.v: UART 发送器	38
4.3.1	设计目标	38
4.3.2	状态机设计	38
4.3.3	源码与讲解	38
4.4	uart_rx.v: UART 接收器	41
4.4.1	设计目标	41

4.4.2	状态机设计	42
4.4.3	源码与关键思路讲解	42
4.5	综合到 FPGA: 从代码到硬件	45
4.5.1	从 Verilog 到真实硬件的流程	45
4.5.2	u_lite 程序如何加载到 FPGA 的指令存储器	46
4.5.3	和 QEMU 模拟的对应关系	46
4.5.4	为什么要做这种事	47
5	C 语言算法与数据结构	48
5.1	本章讲什么	48
5.2	插入排序—逐个插入	48
5.2.1	核心思路	48
5.2.2	时间复杂度	49
5.2.3	C 代码	49
5.2.4	代码解读	49
5.3	归并排序—分治 + 逆序对	50
5.3.1	核心思路	50
5.3.2	时间复杂度	50
5.3.3	C 代码—合并函数	51
5.3.4	C 代码—递归排序函数与主程序	52
5.3.5	编译与运行	52
5.4	堆排序—把数组当二叉树	53
5.4.1	核心思路: 把数组看成一棵二叉树	53
5.4.2	C 代码—堆性质维护 Maxheapify	53
5.4.3	C 代码—建堆 BuildmaxHeap	54
5.4.4	C 代码—Heapsort 主函数	54
5.4.5	C 代码—辅助函数与测试	55
5.4.6	C 代码—main 测试程序	55
5.4.7	时间复杂度	56
5.4.8	与其他排序的对比	56
5.5	最大子数组—从暴力到分治	56
5.5.1	解法一: 暴力枚举所有可能的子数组	56
5.5.2	解法二: 分治法	58
5.5.3	时间复杂度对比	60
5.6	矩阵乘法—二维分治	61
5.6.1	数据结构	61
5.6.2	C 代码—创建、释放、打印矩阵	61
5.6.3	C 代码—补零到 2 的幂次方	62
5.6.4	C 代码—拆分、合并与矩阵加法	63

5.6.5	C 代码—递归乘法主体	64
5.6.6	C 代码—主程序验证	66
5.6.7	从底层视角看矩阵乘法	67
5.7	从 C 代码到汇编—底层视角	67
5.7.1	函数调用 vs 栈操作	67
5.7.2	数组与指针 vs [base, index, shift]	67
5.7.3	结构体与内存对齐	68
5.7.4	从算法到硬件的路径	68

Chapter 1

项目介绍

这是一个从底层向上构建的学习项目。我们不依赖高级语言编译器和操作系统提供的便利，而是亲自去触碰：

- **芯片** (Chip)：用 Verilog 写电路，从简单的字节加法器，到 UART 串口收发器，再到自制 CPU
- **系统** (ISA + QEMU)：用 AArch64 汇编直接写用户态程序，然后把代码加载到 QEMU 仿真的裸机环境里运行
- **算法**：把学到的底层能力，用 C 实现标准算法和数据结构
- **AI**：用 CUDA 并行计算，体会 GPU 与 CPU 协作的底层原理

整个项目的核心哲学是：自己动手做一遍，才真正理解了这一层在做什么。

1.1 目录结构

项目顶层目录组织如下：

ISA/ 用户态汇编程序区。它包含一个完整的 AArch64 汇编程序：手工实现了堆内存分配器、字符串管理、命令解析器。所有 I/O 都是通过 ARM 的 `svc` 指令直接触发系统调用。主要文件：

- `hello.s` —主程序入口，循环读取、复制、存储、打印用户输入
- `src/exit.s` —`sys_write`、`sys_read`、`exit` 的系统调用封装
- `src/memory.s` —`memory_init`、`memory_alloc`，一个 4 KB 堆的分配器
- `src/memoryfree.s` —`memory_free`，带相邻块合并
- `src/memorycopy.s` —`copy_string`，从输入缓冲区复制到新分配的内存
- `src/string_array.s` —平行数组存储字符串指针与长度，按序号存取
- `src/str_parse_cmd.s` —`find_two_spaces`，扫描字符串找到前两个空格的位置
- `src/test.s` —测试函数：内存读写、系统调用的测试
- `src/memory.inc` —常量定义（如 `HEAP_SIZE = 4096`）

- `run.sh` —— 一键编译并启动 lldb 调试

QEMU/ 裸机 AArch64 程序区。不使用任何操作系统，直接在 QEMU 仿真的 virt 机器上运行，通过内存映射 I/O 访问 PL011 UART 输出字符串。主要文件：

- `start.s` — 启动代码：设置 UART 基地址，逐字节输出，然后进入 `wfe` 等待
- `run.sh` — 用 Homebrew 的 clang/LLVM 交叉编译、链接、启动 QEMU
- `bin/kernel.elf` — 编译产物（链接地址 `0x40000000`）

Chip/ Verilog 硬件设计区。用寄存器传输级（RTL）代码描述硬件电路。主要文件：

- `src/adder.v` — 64 位字节加法器：把 8 个字节逐位相加，验证移位与位提取
- `src/uart_rx.v` — UART 接收器：5 状态状态机，按波特率采样 RX 引脚
- `src/uart_tx.v` — UART 发送器：4 状态状态机，逐位把字节送到 TX 引脚
- `design.md` — 硬件设计思路文档

1.2 阅读顺序建议

如果你从底层开始学习，推荐的阅读顺序是：

1. 先读本章的 **ISA**（第 2 章）：你已经在 macOS 上跑过用户态汇编，从系统调用和内存管理入手最自然。
2. 再读 **QEMU 裸机**（第 3 章）：体会没有操作系统时，程序是怎么与硬件直接通信的。
3. 最后读 **Chip 硬件设计**（第 4 章）：当你理解了软件如何使用硬件，就可以反过来设计硬件本身了。

如果你对硬件更感兴趣，也可以从第 4 章入手，再回头看软件层是怎么调用它的。

Chapter 2

用户态 AArch64 汇编

本章我们从零开始，用 AArch64（ARM 64 位）汇编语言在 macOS 上写一个完整的用户态程序。它没有 `main` 函数，没有 `printf`，也没有 `malloc`——所有这些我们都用最底层的方式自己实现。

程序的整体结构是：一个 4 KB 的堆内存分配器，一个按序号存储字符串的平行数组，以及一个命令解析器的雏形。主程序循环工作流程为：

读取输入 → 分配内存并复制 → 存入字符串数组 → 打印 → 扫描空格位置

2.1 系统调用：最底层的 I/O

2.1.1 为什么需要系统调用

在用户态运行的程序，不能直接操作硬件。当我们想在屏幕上输出、从键盘读入，或者结束程序，都必须请求内核代劳。这种请求就叫做**系统调用（system call）**。

在 ARM 64 位架构下，系统调用通过 `svc #0x80` 指令触发。操作系统收到这个中断后，根据 `x16` 寄存器里的编号，执行不同的操作。

2.1.2 `exit.s`：三个系统调用的封装

整个程序的所有 I/O 都由下面这几十个字节的代码完成：

```
1 .global exit
2 .global sys_write
3 .global sys_read
4 .align 4
5
6 exit:
7     movz x16, #0x1
8     movk x16, #0x200, lsl #16
9     svc #0x80
10    ret
11
12 sys_write:
13     movz x16, #0x4
14     movk x16, #0x200, lsl #16
15     svc #0x80
16     ret
17
```



```

18 sys_read:
19     movz x16, #0x3
20     movk x16, #0x200, lsl #16
21     svc #0x80
22     ret

```

Listing 2.1: ISA/src/exit.s ——sys_write / sys_read / exit

2.1.3 代码解读

编号拼接：movz + movk 每个系统调用需要把编号放入 x16。ARM 上不能一步写入超过 16 位的立即数，所以使用两步：

- movz x16, #0x1 ——把 x16 的低 16 位置为 0x1，其余清零
- movk x16, #0x200, lsl #16 ——保持其他位不变，把 16 位以上的部分写入 0x200

合起来就是 0x2000001 (exit)、0x2000003 (read)、0x2000004 (write)，这是 macOS XNU 内核的系统调用编号约定。

参数传递 三个函数都没有显式的参数处理，因为它们遵循 AArch64 的调用约定：调用者把参数放在 x0、x1、x2 里，sys_write / sys_read / exit 自己不需要再做什么——只要把编号放到 x16，然后 svc #0x80 就够了。内核会自动从 x0-x2 取走参数。

举个例子——调用 sys_write 向标准输出写一个字符串：

```

1     mov x0, #1           // 文件描述符：1 = 标准输出
2     adrp x1, msg@PAGE    // 字符串所在页的基地址
3     add x1, x1, msg@PAGEOFF // 加页内偏移，得到真正的地址
4     mov x2, #msg_len     // 字符串长度
5     bl sys_write         // 调用 sys_write (bl = branch with link)

```

Listing 2.2: sys_write 的典型调用

svc #0x80：真正的内核切换 这条指令的作用是：产生一个异常，CPU 从用户态切换到内核态，跳转到内核中预先注册的异常向量表。内核根据 x16 的值执行对应处理函数，完成后再返回用户态，继续执行下一条 ret。如果没有这条指令，我们写的程序根本无法和外部世界交互。

2.2 堆内存分配器

2.2.1 设计思路

我们需要一个比 .space 伪指令更灵活的内存分配方式——可以按需分配、用完释放、释放的空间再利用。

我们的实现是一个简化版的 K&R 风格分配器：

1. 在数据段预留一块 4096 字节的连续内存作为堆
2. 每个内存块有一个 8 字节的头：前 4 字节存大小，后 4 字节存是否在用
3. 分配时从堆头开始扫描，找第一个足够大的空闲块
4. 释放时把块标记为空闲，并与左右相邻空闲块合并（coalescing）

一个块的内存布局：

size (4 字节)	used (4 字节)	实际数据区 (size 字节)
8 字节头		

整个堆的结构（假设分配了两块，中间一块空闲）：

size 20	used 1	data 20B	size 100	used 0	data 100B (空闲)	data 40B
---------	--------	----------	----------	--------	----------------	----------

2.2.2 heap 区的定义与初始化

在 `hello.s` 的 `.data` 段里，我们这样定义堆：

```

1 .include "src/memory.inc"      ; 定义 HEAP_SIZE = 4096
2 .data
3 .align 4
4
5 .global heap
6 .global heap_end
7
8 heap:                          ; 堆内存的开始位置
9     .space HEAP_SIZE          ; 开辟一块长度为 4096 字节的内存空间
10 heap_end:                     ; 堆内存的结束位置

```

Listing 2.3: `hello.s` 中的 heap 定义

这里的 `.space` 是汇编器的伪指令：它在目标文件里预留空间，不产生任何机器码。运行时这就是进程虚拟地址空间中的一块内存。

注意我们还定义了两个缓冲区：

```

1 .global input_buf
2 input_buf: .space 256          ; 存放键盘读入的内容
3
4 .global output_buf
5 output_buf: .space 256         ; 存放准备输出的内容

```

Listing 2.4: `hello.s` 中的 I/O 缓冲区

这两块 256 字节的内存，分别对应命令行输入和程序输出。

2.2.3 memory_init: 初始化堆

分配器初始化只做一件事：在堆的第一个 8 字节头部填入整个堆的大小和“空闲”标记。

```

1 .include "src/memory.inc"
2 .extern heap
3 .extern heap_end
4 .global memory_init
5 .global memory_alloc
6
7 memory_init:
8     adrp x0, heap@PAGE           ; 拿到 heap 所在页的基地址
9     add x0, x0, heap@PAGEOFF     ; 加页内偏移, 得到 heap 真正的虚拟地址
10    mov x1, #HEAP_SIZE           ; x1 = 堆总大小 = 4096
11    sub x1, x1, #8               ; x1 = 可用空间 = 4096 - 8 = 4088
12    str w1, [x0]                 ; 把可用大小写入 head 的第一个字 (size 字段)
13    mov w2, #0
14    str w2, [x0, #4]             ; 第二字 (used 字段) 写 0, 表示空闲
15    ret

```

Listing 2.5: memory.s —memory_init

执行完 memory_init, 堆的状态:

4088	0	(未初始化, 4088 字节空闲)
第一个块的头部		

地址计算的关键: adrp + add ARM 64 位下不能用一条指令直接写一个 64 位绝对地址。adrp 把 heap 所在的 4 KB 页的基地址写入 x0, 然后 add 再加上页内偏移——两步合起来就得到了真正的虚拟地址。这是位置无关代码 (PIC) 的标准做法。

2.2.4 memory_alloc: 分配一块内存

这是分配器的核心。它的思路是顺序扫描 (first-fit):

```

1 memory_alloc:
2     ; 入栈保护: 保存 x19-x22 (调用者保存寄存器)
3     stp x19, x20, [sp, #-32]!
4     stp x21, x22, [sp, #16]
5     mov x19, x0                 ; x19 = 申请的大小
6     adrp x20, heap@PAGE
7     add x20, x20, heap@PAGEOFF  ; x20 = 扫描指针 (从 heap 头部开始)
8     adrp x21, heap_end@PAGE
9     add x21, x21, heap_end@PAGEOFF
10
11 alloc_loop:
12     cmp x20, x21                ; 超过堆尾?

```

```

13      b.hs alloc_fail                ; 是 → 失败
14
15      ldr  w22, [x20]                ; 读当前块的 size
16      ldr  w23, [x20, #4]           ; 读当前块的 used 标记
17
18      cmp  w23, #1                   ; 当前块在用吗?
19      b.eq next_block               ; 是 → 跳过
20      cmp  w22, w19                 ; 当前块大小 >= 需要的大小?
21      b.lt next_block               ; 不够 → 跳过

```

Listing 2.6: memory.s —memory_alloc (第一部分：扫描循环)

一旦找到一块合适的空闲块，我们需要决定是否把它**切割**成两块——一块分配出去，剩下的留作新的空闲块。只有剩余空间足够容纳一个头部（8 字节）+ 至少 1 字节数据时才有切割的意义。

```

1      ; 找到了：先标记为已用
2      mov  w0, #1
3      str  w0, [x20, #4]            ; used = 1
4
5      sub  x1, x22, x19              ; x1 = 剩余 = 当前块大小 - 要分配的大小
6      cmp  x1, #8                   ; 剩余小于 8 字节?
7      b.le no_split                ; 是 → 不切割，整块都分配
8
9      ; 切割：在剩余位置写入一个新空闲块的头部
10     add  x2, x20, #8
11     add  x2, x2, x19               ; x2 = 新块头部 = 当前块头 + 8 + 分配大小
12     sub  x3, x1, #8                ; x3 = 新块可用字节 = 剩余 - 8 (新块头)
13     str  w3, [x2]                  ; 新块 size = x3
14     mov  w4, #0
15     str  w4, [x2, #4]              ; 新块 used = 0 (空闲)
16
17     str  w19, [x20]                ; 更新当前块 size = 申请的大小
18
19 no_split:
20     add  x0, x20, #8               ; 返回值 = 数据区首地址 (头 + 8)
21     b     alloc_done
22
23 next_block:
24     add  x20, x20, #8              ; 跳过头部
25     add  x20, x20, x22             ; 跳过数据区, x20 指向了下一个块
26     b     alloc_loop
27
28 alloc_fail:
29     mov  x0, #0                    ; 返回 0 (NULL), 表示分配失败
30
31 alloc_done:
32     ldp  x21, x22, [sp, #16]

```

```

33     ldp x19, x20, [sp], #32      ; 恢复寄存器, 同时释放栈帧
34     ret

```

Listing 2.7: memory.s —memory_alloc (第二部分: 分配与切割)

为什么要保护 x19–x22? AArch64 的调用约定 (AAPCS64) 把寄存器分成两组:

- **临时寄存器** x0--x18: 函数可以随使用, 用完不需要恢复。调用者如果还要用, 必须自己保存。
- **被调用者保存寄存器** x19--x28: 函数如果要用它们, 必须在函数返回前恢复原值。

memory_alloc 用了 x19--x22 来保存扫描指针和大小, 所以进入函数时先把它们压栈, 退出前恢复。这是标准的函数序言/尾声。

2.2.5 memory_free: 释放内存并合并相邻空闲块

释放的逻辑是: 把块标记为空闲, 然后检查左右邻居是否也是空闲, 如果是就把它合并成一块 (否则堆会被切成越来越碎的小块)。

```

1  .include "src/memory.inc"
2  .extern heap
3  .extern heap_end
4  .global memory_free
5
6  memory_free:
7      stp x19, x20, [sp, #-96]!
8      stp x21, x22, [sp, #16]
9      stp x23, x24, [sp, #32]
10     stp x25, x26, [sp, #48]
11     stp x27, x28, [sp, #64]
12     stp x29, x30, [sp, #80]
13
14     ; 第一步: 标记当前块为空闲
15     cbz x0, free_done      ; NULL 指针直接返回
16     sub x19, x0, #8        ; x19 = 当前块头部
17     mov w1, #0
18     str w1, [x19, #4]      ; used = 0
19
20     ; 准备扫描指针, 向前一块找
21     adrp x20, heap@PAGE
22     add x20, x20, heap@PAGEOFF
23     adrp x21, heap_end@PAGE
24     add x21, x21, heap_end@PAGEOFF
25     mov x22, x20
26     mov x23, #0
27

```

```

28 find_prev_loop:
29     cmp x22, x19                ; 扫描到了当前块位置?
30     b.hs check_prev            ; 是 → 跳出循环, 此时 x23 就是前一块
31     cmp x22, x21                ; 超过堆尾?
32     b.hs check_prev
33     mov x23, x22                ; 记录"上一块"的位置
34     ldr w24, [x22]
35     add x22, x22, #8
36     add x22, x22, x24           ; 跳到下一个块的头
37     b find_prev_loop

```

Listing 2.8: memoryfree.s —memory_free

循环结束后, x23 里存的是前一块的地址 (如果当前块是第一个块, x23 保持为 0)。
接下来检查前后两个邻居:

```

1 check_prev:
2     cbz x23, check_next        ; 没有前一块 → 跳过
3     ldr w25, [x23, #4]
4     cmp w25, #0                ; 前一块是空闲吗?
5     b.ne check_next           ; 不空闲 → 跳过
6     ; 合并前一块: 把前一块的 size 扩大, 把当前块吃掉
7     ldr w24, [x23]             ; 前一块 size
8     ldr w26, [x19]             ; 当前块 size
9     add w24, w24, w26
10    add w24, w24, #8            ; +8 吃掉当前块头部
11    str w24, [x23]             ; 写回前一块的 size
12    mov x19, x23               ; 现在"当前块"就是合并后的那块
13
14 check_next:
15    ldr w26, [x19]
16    add x27, x19, #8
17    add x27, x27, w26           ; x27 = 当前块后面那一块的头
18    cmp x27, x21               ; 后一块超过堆尾?
19    b.hs free_done             ; 是 → 没东西合并
20
21    ldr w28, [x27, #4]
22    cmp w28, #0                ; 后一块空闲吗?
23    b.ne free_done
24    ; 合并后一块: 把后一块的 size 加到当前块
25    ldr w29, [x27]
26    add w26, w26, w29
27    add w26, w26, #8
28    str w26, [x19]
29
30 free_done:
31    ; 恢复所有保存过的寄存器 (与进入时的 stp 顺序相反)
32    ldp x29, x30, [sp, #80]

```

```

33     ldp x27, x28, [sp, #64]
34     ldp x25, x26, [sp, #48]
35     ldp x23, x24, [sp, #32]
36     ldp x21, x22, [sp, #16]
37     ldp x19, x20, [sp], #96
38     ret

```

Listing 2.9: memoryfree.s —合并相邻块

释放一个块时，最多可能合并三块（前一块 + 当前块 + 后一块）。正因为我们扫描的方式——从堆头一路数块大小、逐个跳过——才能正确找到“当前块的前一块”的头位置。

2.2.6 测试内存分配器

test.s 里有一个专门测试内存系统的函数：

```

1 test_memory:
2     stp x29, x30, [sp, #-16]!
3     sub sp, sp, #32                ; 留出 32 字节栈空间存指针
4
5     mov x0, #8
6     bl memory_alloc                ; 第一次分配 8 字节
7     str x0, [sp, #0]               ; 存返回的指针到栈上
8     ldr w4, =0x11111111
9     ldr w5, =0x11111111
10    str w4, [x0]                    ; 写入测试数据
11    str w5, [x0, #4]
12
13    mov x0, #8
14    bl memory_alloc                ; 第二次分配（验证切割是否正常）
15    str x0, [sp, #8]
16    ldr w4, =0x22222222
17    ...
18
19    mov x0, #8
20    bl memory_alloc                ; 第三次分配
21    str x0, [sp, #16]
22    ldr w4, =0x33333333
23    ...
24
25    ldr x0, [sp, #16]
26    bl memory_free                  ; 先释放第三块
27    str xzr, [sp, #16]
28
29    ldr x0, [sp, #8]
30    bl memory_free                  ; 再释放第二块（触发与第三块合并）
31    str xzr, [sp, #8]

```

```

32
33     add sp, sp, #32
34     ldp x29, x30, [sp], #16
35     ret

```

Listing 2.10: test.s —test_memory

这个测试的意义是：如果分配、切割、释放、合并这四个步骤都正确工作，堆里的状态就会经历”三块在用”→”第一块在用，后两块释放并合并”的过程。后续再分配一个大请求时，就会从合并后的大块里切分——这正是分配器的正常工作方式。

2.3 字符串复制：用户态的 strcpy

2.3.1 copy_string：从 input_buf 复制到新分配的堆内存

我们用 `sys_read` 把键盘输入读入 `input_buf`，但这块缓冲区是全局共享的，下一次读入会覆盖。因此必须把当前读到的内容复制一份到新分配的内存里。

```

1  .include "src/memory.inc"
2  .extern heap
3  .extern heap_end
4  .extern input_buf
5  .extern output_buf
6  .extern memory_alloc
7  .global copy_string
8
9  copy_string:
10     stp x29, x30, [sp, #-32]!
11     stp x19, x20, [sp, #16]
12
13     mov x19, x0                ; x19 = 输入字节数 (sys_read 的返回值)
14     sub x19, x19, #1           ; 减 1, 把换行符排除
15
16     ; 先分配一块大小为 x19 的内存
17     adrp x20, input_buf@PAGE
18     add x20, x20, input_buf@PAGEOFF
19     bl memory_alloc            ; 返回值 x0 是新内存块的 data 区首地址
20
21     mov x2, #0                 ; 循环计数器
22
23 copy_loop:
24     cmp x2, x19                ; 是否已复制全部字节?
25     b.ge copy_done
26     ldrb w4, [x20, x2]         ; 从 input_buf 读一个字节
27     strb w4, [x0, x2]         ; 写到新内存
28     add x2, x2, #1

```



```

29     b    copy_loop
30
31 copy_done:
32     add x19, x19, #1
33     add x2, x2, #1
34     strb wzr, [x0, x19]      ; 在末尾写一个 0 字节 (C 字符串结尾)
35
36     ldp x19, x20, [sp, #16]
37     ldp x29, x30, [sp], #32
38     ret

```

Listing 2.11: memorycopy.s —copy_string

函数输入输出 调用者把 `sys_read` 的返回值（实际读取的字节数）放到 `x0`，然后 `bl copy_string`。返回时 `x0` 指向一块新分配的内存，里面是用户输入的内容，末尾加了一个 0 字节。

为什么要手动加 `wzr`? `sys_read` 只是把字节原样从终端拷到缓冲区，不做任何处理。它不会给你加字符串结尾标记。所以 `strb wzr, [x0, x19]` 这一步是在告诉任何后续的“按字符串方式处理”的代码：到这里为止。

关于 `x19` 减 1 又加 1 的设计

- 进入时 `sys_read` 返回的字节数包含末尾的换行
- `sub x19, x19, #1` 把换行符排除在复制循环之外
- 但在写入终止 0 时，用的是原来的 `x19+1` 的位置（也就是换行符所在的位置），用 0 覆盖了它

这样最终得到的字符串既不包含换行，又是一个合法的 C 风格字符串。

2.4 平行数组：字符串的索引存储

2.4.1 为什么需要索引存储

每一次循环，用户都会输入一行内容。如果只是把它们零散地放到堆里，时间久了就找不到了。我们需要一种**按序号**存取的结构：存第 0 个、第 1 个、第 2 个……，然后按同样的序号取出来打印。

为此我们用两个平行数组：一个存每个字符串的**指针**（64 位 / 8 字节），一个存每个字符串的**长度**（32 位 / 4 字节）。配合一个计数器记录存了多少条。

2.4.2 string_array.s: 数据区定义

```

1 .data
2 .align 4
3
4 .equ MAX_STRINGS, 32          ; 最多存 32 条字符串
5
6 string_count: .word 0          ; 计数器，已存入多少条
7 string_ptrs: .space MAX_STRINGS * 8 ; 指针数组，每项 8 字节
8 string_lens: .space MAX_STRINGS * 4 ; 长度数组，每项 4 字节

```

Listing 2.12: string_array.s —数据区定义

从内存地址看，它的布局是：

string_count	4 字节	存放当前已存入条数	
string_ptrs[0]	8 字节	第 0 条的地址	
string_ptrs[1]	8 字节	第 1 条的地址	
⋮			
string_ptrs[31]	8 字节	第 31 条的地址	
string_lens[0]	4 字节	第 0 条的长度	
⋮			

2.4.3 store_string_ptr_len: 写入一对指针和长度

```

1 .text
2 .global store_string_ptr_len
3 .global store_ptr_done
4 .global get_string_ptr
5 .global get_string_len
6 .global print_index_strings
7 .extern sys_write
8
9 store_string_ptr_len:
10     stp x29, x30, [sp, #-16]!
11     mov x29, sp
12
13     ; 检查是否超过上限
14     cmp x1, #MAX_STRINGS
15     b.hs store_ptr_done
16
17     ; 存指针: string_ptrs[x1 * 8] = x0
18     adrp x4, string_ptrs@PAGE
19     add x4, x4, string_ptrs@PAGEOFF
20     str x0, [x4, x1, lsl #3] ; x1 * 8 作为偏移

```

```

21
22 ; 存长度: string_lens[x1 * 4] = x2
23 adrp x3, string_lens@PAGE
24 add x3, x3, string_lens@PAGEOFF
25 lsl x5, x1, #2
26 str w2, [x3, x5]
27
28 store_ptr_done:
29 ldp x29, x30, [sp], #16
30 ret

```

Listing 2.13: string_array.s —store_string_ptr_len

调用约定 调用者需要把三个参数放进来：

- x0 —字符串在内存中的起始地址（就是 `copy_string` 的返回值）
- x1 —序号（第几条）
- x2 —字符串长度（字节数）

注意 ARM 的指令 `str x0, [x4, x1, lsl #3]` 可以直接在一个内存操作里完成“基址 + 变址 × 8”的计算——因为指针是 8 字节，正好用 `lsl #3`（左移 3 位 = ×8）。长度是 4 字节，所以用 `lsl #2`（左移 2 位 = ×4），但 AArch64 不允许 `str` 的寻址直接放 `lsl #2`，所以先算到 x5 再用。

2.4.4 print_index_strings: 按序号把字符串打印出来

写入之后我们要能读回来打印。读的逻辑是写入的反向：

```

1 print_index_strings:
2     stp x29, x30, [sp, #-16]!
3     mov x29, sp
4
5     cmp x0, #MAX_STRINGS
6     b.hs print_index_strings_done
7
8     ; 取指针: x1 = string_ptrs[x0 * 8]
9     adrp x3, string_ptrs@PAGE
10    add x3, x3, string_ptrs@PAGEOFF
11    ldr x1, [x3, x0, lsl #3]
12
13    ; 取长度: x2 = string_lens[x0 * 4]
14    adrp x3, string_lens@PAGE
15    add x3, x3, string_lens@PAGEOFF
16    lsl x4, x0, #2
17    ldr w2, [x3, x4]

```

```

18
19 ; 如果指针或长度为 0, 说明这个位置从未被写入过
20 cbz x1, print_index_strings_done
21 cbz x2, print_index_strings_done
22
23 ; 调用 sys_write(1, ptr, len) 打印
24 mov x0, #1
25 stp x1, x2, [sp, #-16]! ; 压栈保护 x1,x2
26 bl sys_write
27 ldp x1, x2, [sp], #16
28
29 print_index_strings_done:
30 ldp x29, x30, [sp], #16
31 ret

```

Listing 2.14: string_array.s —print_index_strings

调用者只需要把序号 (0、1、2…) 放 x0, bl print_index_strings 就能把那一行打印到标准输出。如果那个序号的位置从未写入过, string_ptrs[i] 里是 0 (.space 初始化为 0), 就会被 cbz 抓住, 直接退出——不会因为野指针而崩溃。

为什么用两个数组而不是一个结构体数组? 如果用结构体 (每条存一个指针 + 一个长度, 12 字节对齐到 16 字节), 每次写入都要写一条连续的 16 字节, 而读的时候不需要长度的话也要把整条读出来。分开两个数组时:

- 写入: 两条 store 指令
- 只需要指针时: 只读 8 字节
- 只需要长度时: 只读 4 字节
- 按序号索引时, 不需要算结构体对齐的 padding

对于这个小程序来说两种方式都可以, 但平行数组的写法更直观, 循环内每次只做一件小事。

2.5 命令解析: find_two_spaces

2.5.1 把命令切成 token 的第一步

如果用户输入的是 "add file1 file2" 之类的命令, 我们需要把它切开: 动词是第一个词, 参数是第二、第三个词……最朴素的方法就是扫一遍字符串, 找出**第一个空格**和**第二个空格**的位置, 然后两个空格之间的就是各个 token。

2.5.2 find_two_spaces: 单遍扫描找两个空格

```

1 .text
2 .global find_two_spaces
3
4 find_two_spaces:
5     stp x29, x30, [sp, #-16]!
6     mov x29, sp
7
8     mov x0, x1           ; x0 = 字符串起始地址
9     mov x1, x2           ; x1 = 字符串总长度
10    mov x2, #0           ; x2 = 当前扫描位置
11    mov x3, #-1          ; x3 = 第一个空格的位置 (-1 = 未找到)
12    mov x4, #-1          ; x4 = 第二个空格的位置
13
14 loop:
15     cmp x2, x1           ; 已扫描完?
16     b.hs not_found_check
17
18     ldrb w5, [x0, x2]    ; 读一个字节
19     cmp w5, #' '        ; 是空格吗?
20     b.ne next_byte
21
22     cmp x3, #-1          ; 第一个空格还没记录?
23     b.ne found_second
24     mov x3, x2           ; 是第一个空格
25     b next_byte
26
27 found_second:
28     mov x4, x2           ; 找到第二个空格
29     b done
30
31 next_byte:
32     add x2, x2, #1
33     b loop
34
35 not_found_check:
36     cmp x3, #-1
37     b.eq not_found       ; 连第一个空格都没找到
38     cmp x4, #-1
39     b.eq not_found       ; 只找到一个空格
40     b done
41
42 not_found:
43     mov x0, #-1
44     mov x1, #-1
45     b return

```

```

46
47 done:
48     mov x0, x3           ; x0 = 第一个空格位置
49     mov x1, x4           ; x1 = 第二个空格位置
50
51 return:
52     ldp x29, x30, [sp], #16
53     ret

```

Listing 2.15: str_parse_cmd.s —find_two_spaces

返回值约定 调用 find_two_spaces 后：

- 成功找到两个空格：x0 = 第一个空格的偏移，x1 = 第二个空格的偏移
- 失败：x0 = -1, x1 = -1

有了这两个位置，后续就可以做简单切分：

- 从 0 到 x0 之间 → 第一个 token（如”add”）
- 从 x0+1 到 x1 之间 → 第二个 token（如”file1”）
- 从 x1+1 到末尾 → 第三个 token（如”file2”）

注意 AArch64 下的寻址限制 代码里用 ldrb w5, [x0, x2]，这里的 x2 是普通寄存器，可以直接作为变址。但在 store_string_ptr_len 里，我们用 str w2, [x3, x5] 而不是 str w2, [x3, x1, lsl #2]——因为 AArch64 的”寄存器 + 寄存器 × 缩放”寻址只允许缩放因子为 0、1、2、3（即 ×1、×2、×4、×8），但 w2（32 位寄存器）作为目的时，str w2 的指令编码里不允许同时用带 lsl 的变址。所以当缩放因子不是 3 时，我们先把 x1 * 4 算到 x5 里再用。

2.6 主程序：把一切串起来

2.6.1 hello.s：主程序入口

主程序的工作非常简单——初始化堆之后，就进入一个循环：读一行 → 复制 → 存到平行数组 → 打印 → 找空格。

```

1 .text
2 .global _main
3 .extern exit
4 .extern memory_init
5 .extern memory_alloc
6 .extern memory_free

```

```

7 .extern sys_write
8 .extern sys_read
9 .extern test_write
10 .extern test_memory
11 .extern test_read
12 .extern test_write_memory
13 .extern store_string_ptr_len
14 .extern get_string_ptr
15 .extern get_string_len
16 .extern store_ptr_done
17 .extern print_index_strings
18 .extern find_two_spaces
19
20 _main:
21     bl memory_init           ; 初始化堆
22     mov x19, #0             ; x19 = 循环计数器 (也是字符串序号)
23
24 loop:
25     mov x20, #0
26     bl store_string         ; 读一行, 分配内存, 存入数组
27
28     mov x20, #0
29     bl get_string_ptr_len_spaces ; 顺便检查空格位置
30
31     add x19, x19, #1
32     cmp x19, #1
33     b.eq do_exit
34     b loop
35
36 do_exit:
37     b exit

```

Listing 2.16: hello.s —主程序

2.6.2 store_string: 读入并保存一行

```

1 store_string:
2     stp x29, x30, [sp, #-16]!
3     bl test_read           ; 把一行
4     bl copy_string         ; copy_string(x0) 从 input_buf 复制到新分配的堆内存
5     mov x1, x20             ; 序号
6     bl store_string_ptr_len ; 存入平行数组
7     ldp x29, x30, [sp], #16
8     ret

```

Listing 2.17: hello.s —store_string

2.6.3 get_string_ptr_len_spaces: 打印并解析空格

```

1 get_string_ptr_len_spaces:
2     stp x29, x30, [sp, #-16]!
3
4     bl print_string           ; 把刚存入的字符串再取出来打印
5     stp x1, x2, [sp, #-16]! ; 保存打印返回的指针和长度
6     bl find_two_spaces       ; 然后扫描字符串找空格位置
7     ldp x2, x3, [sp], #16
8
9 get_string_ptr_len_spaces_done:
10    ldp x29, x30, [sp], #16
11    ret

```

Listing 2.18: hello.s —打印 + 解析

2.6.4 理解整体数据流

整个程序的数据流如下：

```

stdin → sys_read → input_buf → copy_string(从 input_buf 到 heap)
      → (ptr, len) → store_string_ptr_len
      → print_index_strings(x19) → sys_write → stdout
      → find_two_spaces(buffer) → x0=第一个空格, x1=第二个空格

```

程序循环一次（x19 递增一，直到 x19 = 1 时退出。你可以把 `cmp x19, #1` 改成更大的数字来延长程序执行更多轮。

2.6.5 主循环限制

这里的循环在 x19 计数，最多能存 MAX_STRINGS (32) 条字符串。超过会被 store_string_ptr_len 里的 `cmp x1, #MAX_STRINGS; b.hs store_ptr_done` 拦截掉——如果 x19 超过 32，函数什么都不做就返回。

2.6.6 新版主程序：把解析后的 token 逐条打印

在引入 store_instr_addrs 之后，主程序多了一步：把刚才那条命令的 4 个 token 分别打印出来，验证解析是否正确。

```

1 _main:
2     bl memory_init           ; 初始化堆
3     mov x19, #0              ; x19 = 循环计数器
4
5 loop:
6     mov x20, #0

```



```

7      bl store_string           ; 读一行 → 复制 → 存到平行数组
8
9      mov x20, #0
10     bl get_string_ptr_len_spaces ; 顺便找两个空格位置, 顺便把字符串打印出来
11                                     ; 返回后: x0 = 第一个空格, x1 = 第二个空格
12                                     ; x2 = 字符串起始, x3 = 字符串长度
13
14     bl store_instr_addrs       ; 把刚才的一行切成 3 个 token, 存进 instr_addrs
15
16     ; ---- 把 token 0/1/2/3 分别打印出来, 每个后面换行 ----
17     mov x0, #0
18     bl get_instr_addrs        ; x0 = 第 0 条指令的 4 指针数组头
19     bl get_instr_0            ; 打印 token 0
20     bl print_newline
21
22     mov x0, #0
23     bl get_instr_addrs
24     bl get_instr_1            ; 打印 token 1
25     bl print_newline
26
27     mov x0, #0
28     bl get_instr_addrs
29     bl get_instr_2            ; 打印 token 2
30     bl print_newline
31
32     mov x0, #0
33     bl get_instr_addrs
34     bl get_instr_3            ; 打印 token 3
35     bl print_newline
36
37 flag:
38     add x19, x19, #1
39     cmp x19, #1
40     b.eq do_exit
41     b loop
42
43 do_exit:
44     b exit

```

Listing 2.19: hello.s —新版 __main (核心循环)

调用完空格查找函数后, x0/x1 里是两个空格位置, x2/x3 里是字符串的起始地址和长度, 正好匹配 store_instr_addrs 期望的 4 个入参。

2.6.7 get_instr_0/1/2/3: 从指针数组里拿一个 token 打印

这四个函数的结构几乎一样, 只是读取的偏移不同:

```

1 get_instr_0:
2     stp x29, x30, [sp, #-32]!
3     stp x19, x20, [sp, #16]
4     mov x20, x0                      ; 保存 4 指针数组头
5
6     ; 手动 strlen: 找到 token 0 的长度
7     ldr x1, [x20]                    ; x1 = token 0 的字符串起始
8     mov x2, #0
9 strlen_loop:
10    ldrb w3, [x1, x2]
11    cbz w3, strlen_done
12    add x2, x2, #1
13    b     strlen_loop
14 strlen_done:
15    mov x0, #1
16    bl     sys_write                  ; sys_write(1, token0_ptr, token0_len)
17
18    ldp x19, x20, [sp, #16]
19    ldp x29, x30, [sp], #32
20    ret

```

Listing 2.20: hello.s —get_instr_0 (打印 token 0)

get_instr_1/2/3 的区别只是把 ldr x1, [x20] 换成 ldr x1, [x20, #8]、ldr x1, [x20, #16]、ldr x1, [x20, #24]——分别取 token 1、2、3 的指针。

2.6.8 print_newline: 输出一个换行符

在两个 token 之间打印 \n，让每个 token 单独占一行：

```

1 print_newline:
2     stp x29, x30, [sp, #-32]!
3     strb wzr, [sp, #16]              ; 清一块栈内存
4     mov w8, #10                      ; '\n' 的 ASCII = 10
5     strb w8, [sp, #16]              ; 把换行符写进去
6     add x1, sp, #16                 ; x1 = 换行符所在地址
7     mov x0, #1
8     mov x2, #1
9     bl     sys_write                  ; sys_write(1, "\n", 1)
10    ldp x29, x30, [sp], #32
11    ret

```

Listing 2.21: hello.s —print_newline

它用栈上的一小块空间临时存放换行符，不需要专门准备一个 .data 里的字符串。这是”不依赖静态数据、临时在栈上构造参数”的常见小技巧。

2.6.9 现在整个程序的工作流

综合起来，一次循环做了以下 4 件事：

1. 从 `stdin` 读一行 → `copy_string` 复制到堆 → 把指针和长度存入 `string_ptrs` 与 `string_lens` 两个数组
2. 打印那一行 → 扫描字符串找两个空格位置
3. 用空格位置把一行切成 3 个 token → 存入 `instr_addr`s
4. 按序号把 4 个 token 分别打印出来

这相当于一个最小的”命令解析 + 回显”循环：你输入什么，它就原样打印，再把每个 token 拆出来单独打印一行。

2.7 编译与调试脚本

2.7.1 clang：汇编成可执行文件

整个项目只需要一条命令把所有 `.s` 源文件编译成一个可执行文件：

```
1 #!/bin/zsh
2
3 echo ""
4 echo "  Building hello.s ..."
5 echo ""
6
7 clang -g -o bin/hello hello.s src/*.s -nostdlib -lSystem
8
9 # 然后进入调试器
10 lldb ./bin/hello
```

Listing 2.22: ISA/run.sh ——键构建

各参数含义：

- `-g`：在生成的机器码里插入调试信息（DWARF），让 `lldb` 能显示源文件行号
- `-nostdlib`：不链接默认的 C 标准库。我们自己管理内存
- `-lSystem`：但要链接系统调用库（`libSystem`）。macOS 的 `svc #0x80` 机制在这里面实现
- `hello.s src/*.s`：所有源文件在一条命令里处理
- `lldb ./bin/hello`：启动后立即进入调试器

2.7.2 为什么不用 ld 不用自己链接

一个典型的”手写汇编程序”在 macOS 上其实只需要两件事：汇编和系统调用。**\$clang** 是一个足够智能的汇编器：

- 它自动把你写的 **.s** 文件汇编成机器码
- 自动调用链接器把它们拼起来
- 自动处理系统调用库引用

如果用 **ld** 单独链接，你需要手动指定 **-lSystem** 等库路径，还要保证 **入口点** 符合约定。统一交给 **clang** 处理就省掉了。

2.7.3 调试流程

程序在 LLDB 里，常用命令：

- **b store_string_ptr_len** 或 **b find_two_spaces**：在关键函数上下断点
- **r** 或 **run**：开始运行
- **si**：单步执行一条汇编指令
- **n**：执行下一行（不进入子函数）
- **register read x0**：读寄存器
- **memory read -size 8 -format x -count 16 \$x1**：查看内存内容

要验证分配器是否正确，可以在 **store_string_ptr_len** 上下断点，看 **x0**（字符串地址）、**x1**（序号）、**x2**（长度）是否都正确写入 **string_ptrs[]** 和 **string_lens[]** 中。

2.7.4 运行流程

总结一下：

```
./run.sh
```

- **clang** 汇编、链接所有 **.s**
- **lldb** 启动调试器
- 在程序里输入任意文本
- 读入 → 复制 → 保存 → 打印 → 分析空格

这个过程中，你可以在任何地方打断点调试器看寄存器的值，看内存的内容，观察函数调用栈，确认整个链路的每个环节的数据流动方向。

2.8 指令解析：把命令切成 token 指针数组

`find_two_spaces` 告诉我们” 第一个空格在哪、第二个空格在哪”，但要真正执行命令，我们还需要把各个 **token** 的指针保存下来，后面按序号取回、并做指令比对。这就是 `instr.s` 要做的事：

1. 分配一块内存，里面放 4 个 8 字节指针（最多 4 个 token）
2. 把 [0, 第一个空格)、[第一个空格 +1, 第二个空格)、[第二个空格 +1, 末尾) 三段分别复制到新分配的内存中（每段末尾补 0）
3. 把这 4 个指针的地址放到全局的 `instr_addrs` 数组里，按序号保存，供后续读取

2.8.1 数据区：instr_addrs 全局指针数组

`instr.s` 开头定义了一个与 `string_array.s` 类似的平行数组，区别是每项是一个”指针数组的头”（也就是一块包含 4 个指针的内存）：

```
1 .include "src/memory.inc"
2
3 .data
4 .align 4
5 .extern heap
6 .extern heap_end
7 .equ MAX_INSTR, 64           ; 最多保存 64 条已解析的指令
8 instr_addrs: .space MAX_INSTR*8 ; 每项 8 字节：指向一条 token 数组的指针
9 count_instr: .word 0         ; 当前已存入多少条
10
11 .text
12 .extern memory_init
13 .extern memory_alloc
14 .extern memory_free
15
16 .global store_instr_addrs
17 .global get_instr_addrs
```

Listing 2.23: `instr.s` —数据区定义

布局上，`instr_addrs` 是一个指针数组：第 0 项指向第 0 条指令的 4 个 token 指针块，第 1 项指向第 1 条指令的 4 个 token 指针块，以此类推。
每条指令的”4 个 token 指针块”是这样一块内存：

偏移 0	token_0 的指针（如”add”的首地址）
偏移 8	token_1 的指针（如”file1”的首地址）
偏移 16	token_2 的指针（如”file2”的首地址）
偏移 24	token_3 的指针（保留 / 未使用）

2.8.2 store_instr_addrs: 把一条命令切成 token 并存下

它接收 4 个参数：

- x0 = 第一个空格位置
- x1 = 第二个空格位置
- x2 = 原字符串的起始地址
- x3 = 原字符串的长度

首先检查计数器，然后分配一块 32 字节 (4×8) 的内存用于存放 4 个 token 指针，接着分 3 次把每个 token 复制到新分配的独立内存里，每段末尾补 0 做 C 风格字符串结尾。最后把这块 4 指针的地址存入全局的 `instr_addrs[count_instr]`。

```

1 store_instr_addrs:
2     stp x19, x20, [sp, #-96]!
3     stp x21, x22, [sp, #16]
4     stp x23, x24, [sp, #32]
5     stp x25, x26, [sp, #48]
6     stp x27, x28, [sp, #64]
7     stp x29, x30, [sp, #80]
8
9     mov x19, x0           ; x19 = 第一个空格位置
10    mov x20, x1           ; x20 = 第二个空格位置
11    mov x21, x2           ; x21 = 字符串起始地址
12    mov x22, x3           ; x22 = 字符串总长度
13
14    adrp x25, count_instr@PAGE
15    add x25, x25, count_instr@PAGEOFF
16    ldr w26, [x25]         ; 读当前计数
17    cmp w26, #MAX_INSTR
18    b.hs skip_store       ; 已达上限，跳过
19
20    mov x0, #32
21    bl memory_alloc       ; 分配 32 字节 = 4 个指针
22    mov x24, x0           ; x24 = token 指针数组的头
23
24    adrp x23, instr_addrs@PAGE
25    add x23, x23, instr_addrs@PAGEOFF

```

Listing 2.24: instr.s —store_instr_addrs (入栈与检查)

接下来三段几乎对称的代码分别把 token_0、token_1、token_2 复制出来：

```

1 ; ---- token 0: [0, x19) ----
2 parse_instr1:
3     mov x0, x19           ; 长度 = 第一个空格位置

```

```

4      add x0, x0, #1           ; +1 给结尾 0 留位置
5      bl  memory_alloc        ; 分配新内存存放这一段
6      mov x27, x0             ; x27 = 目标地址
7      mov x28, #0
8  extract_byte_loop1:
9      cmp x28, x19
10     b.ge extract_done1
11     ldrb w29, [x21, x28]
12     strb w29, [x27, x28]
13     add x28, x28, #1
14     b extract_byte_loop1
15 extract_done1:
16     strb wzr, [x27, x28]      ; 末尾补 0
17     str  x27, [x24]           ; token 0 指针写入 instr_addrs[0]
18
19 ; ---- token 1: (x19, x20) ----
20 parse_instr2:
21     sub x0, x20, x19          ; 长度 = 第二个空格 - 第一个空格
22     add x0, x0, #1
23     bl  memory_alloc
24     mov x27, x0
25     mov x28, #0
26     add x29, x19, #1          ; 从第一个空格之后开始读
27 extract_byte_loop2:
28     cmp x29, x20
29     b.ge extract_done2
30     ldrb w30, [x21, x29]
31     strb w30, [x27, x28]
32     add x28, x28, #1
33     add x29, x29, #1
34     b extract_byte_loop2
35 extract_done2:
36     strb wzr, [x27, x28]
37     str  x27, [x24, #8]       ; token 1 指针写入 instr_addrs[1]
38
39 ; ---- token 2: (x20, end) ----
40 parse_instr3:
41     sub x0, x22, x20          ; 长度 = 总长 - 第二个空格
42     add x0, x0, #1
43     bl  memory_alloc
44     mov x27, x0
45     mov x28, #0
46     add x29, x20, #1          ; 从第二个空格之后开始
47 extract_byte_loop3:
48     cmp x29, x22
49     b.ge extract_done3
50     ldrb w30, [x21, x29]

```

```

51     strb w30, [x27, x28]
52     add x28, x28, #1
53     add x29, x29, #1
54     b extract_byte_loop3
55 extract_done3:
56     strb wzr, [x27, x28]
57     str x27, [x24, #16]           ; token 2 指针写入 instr_addrs[2]
58
59     str x27, [x24, #24]         ; token 3 保留 (与 token 2 同值, 方便调试)

```

Listing 2.25: instr.s —解析并复制 3 个 token

最后把这块 4 指针数组的头地址写入全局 `instr_addrs`，计数器加一：

```

1     str x24, [x23, x26, lsl #3] ; instr_addrs[count_instr] = x24
2
3     mov w0, w26                 ; 返回值 = 这条指令的序号
4     add w26, w26, #1
5     str w26, [x25]              ; count_instr++
6
7 instr_addrs_done:
8     ldp x29, x30, [sp, #80]
9     ldp x27, x28, [sp, #64]
10    ldp x25, x26, [sp, #48]
11    ldp x23, x24, [sp, #32]
12    ldp x21, x22, [sp, #16]
13    ldp x19, x20, [sp], #96
14    ret
15
16 skip_store:
17     mov x0, #-1                 ; 失败返回 -1
18     b instr_addrs_done

```

Listing 2.26: instr.s —收尾：写入全局指针数组

2.8.3 get_instr_addrs: 按序号取回 token 指针数组

保存之后，我们希望按“这是第几条指令”把 4 个 token 的指针数组读回来。这是写入的反向操作：

```

1 get_instr_addrs:
2     stp x29, x30, [sp, #-32]!
3     stp x27, x28, [sp, #16]
4
5     mov x28, x0                 ; x28 = 要读取的指令序号
6     adrp x29, instr_addrs@PAGE
7     add x29, x29, instr_addrs@PAGEOFF
8     ldr x0, [x29, x28, lsl #3] ; x0 = instr_addrs[序号]

```



```
9
10 get_instr_addrs_done:
11     ldp x27, x28, [sp, #16]
12     ldp x29, x30, [sp], #32
13     ret
```

Listing 2.27: instr.s —get_instr_addrs

调用它之后, `x0` 就是一块 32 字节的内存, 里面依次放着 `token_0`、`token_1`、`token_2`、`token_3` 的字符串指针。主程序里的 `get_instr_0` / `get_instr_1` / `get_instr_2` / `get_instr_3` 就是把 `x0` 当作“指针数组”来用的例子。

2.8.4 为什么要单独复制 token?

一个很自然的问题: 既然原字符串已经在堆里了, 为什么不直接保存“偏移 + 长度”, 而是额外分配内存把每个 `token` 复制一份?

原因有两个:

1. **C 风格字符串方便后续比对。**后面如果想写 `strcmp("add", token0)` 这种比较, 要求被比较的字符串在自己的末尾处有一个 0 字节。原字符串在 `token` 交界处是空格而不是 0, 所以必须复制一份并补 0。
2. **后续可能修改或释放原字符串。**一旦原字符串所在的行被释放或被覆盖, 我们保存在 `instr_addrs` 里的“指向原字符串内部的指针”就会变成野指针。复制之后 `token` 各自独立, 不再依赖原件。

代价是每条指令会多 3 次 `memory_alloc`, 但我们的堆有 4096 字节, 一个 `token` 通常只有几个字符, 在实验阶段完全够用。

Chapter 3

QEMU 裸机 AArch64

用户态汇编跑在 macOS 内核之上，依赖系统调用完成 I/O。本章我们把这个环境彻底去掉——直接跑在一个“没有任何操作系统”的虚拟 ARM 机器上。

这件事看似疯狂，其实非常简单：你写的汇编程序不再依赖 `sys_write`、`sys_read`，而是直接读写特定内存地址。那个地址上挂了一个 UART 串口设备，你写什么，它就通过串口把字节送到终端；它收到什么，就读回给你。

整个流程：

1. 用 clang (`-target=aarch64-none-elf`) 把 `start.s` 汇编成.o
2. 用 ld.lld 把.o 链接到地址 `0x4000_0000` (QEMU virt 机器的默认载入点)
3. 启动 `qemu-system-aarch64`，让它把我们的 ELF 文件载入到该地址，然后 CPU 从 `_start` 开始执行

3.1 start.s：最简裸机程序

3.1.1 极简的裸机程序

这是一个完整的“在裸机上打印一句话然后永远等待”的程序。

```
1 .section .text
2 .global _start
3
4 _start:
5     ldr x0, =0x09000000      ; UART0 的基地址 (QEMU virt 机器的约定)
6     ldr x1, =msg             ; 字符串所在的地址
7
8 print_loop:
9     ldrb w2, [x1], #1        ; 读 x1 指向的字节，然后 x1++
10    cbz w2, halt             ; 读到 0 (字符串结尾) 就退出
11
12 wait_uart:
13     ldr w3, [x0, #0x18]      ; 读 UART 的 FLAG 寄存器 (UARTFR, 偏移 0x18)
14     tbnz w3, #5, wait_uart   ; bit 5 = TXFF (发送 FIFO 满)，忙等直到不忙
15     str w2, [x0]             ; 把字节写到 UART 的数据寄存器 (UARTDR, 偏移 0x0)
16     b print_loop
17
18 halt:
```

```

19     wfe                                ; 等待事件：进入低功耗状态
20     b     halt                        ; 万一被唤醒，继续等待
21
22 .section .rodata
23 msg:
24     .asciz "Hello QEMU AArch64 bare metal!\n"

```

Listing 3.1: QEMU/start.s —最简裸机程序

这不到 20 行做了下面几件事：

步骤 1：把两个关键地址取到寄存器

- `x0 = 0x0900_0000` ——QEMU 模拟的 ARM PL011 UART0 的 MMIO 基地址。只要往这个地址写数据，QEMU 就会把它当成字符显示到终端。
- `x1 = msg` ——字符串在程序内存中的起始地址。

步骤 2：逐字节打印 每一个字符的处理都要先查询 UART 是否空闲：

- 读 `[x0, #0x18]` (UARTFR 寄存器)，检查第 5 位 (TXFF)。
- TXFF = Transmit FIFO Full。为 1 表示发送缓冲区已满，CPU 必须等待。
- 用 `tbnz w3, #5, wait_uart` 忙等这一位变 0。
- 一旦空闲，`str w2, [x0]` 把当前字节写出去。

PL011 是 ARM 官方定义的 UART 硬件规范，QEMU 按这个标准实现了一个虚拟版本。所以你只要按照 PL011 的寄存器表操作，就能驱动它。

步骤 3：停机 程序打印完字符串（遇到 0 字节即 `.asciz` 的字符串尾）后，执行 `wfe` (Wait For Event)。这是一条 ARM 指令，让 CPU 停止取指、进入低功耗状态，直到外部中断唤醒它。我们没有任何中断源被启用，所以它实际上就是永久停机了。

3.1.2 和 ISA 章节的差异

与 ISA/ 目录里的用户态汇编相比，这段代码有几个不同之处：

- 没有 `ADRP/ADD` 组合——因为我们不要求位置无关代码。链接器会把 `_start` 放到 `0x4000_0000`，所以直接用 `ldr reg, =address` 就能拿到绝对地址。
- 没有 `svc #0x80`——我们直接操作硬件，没有内核帮你做事。
- 入口是 `_start` 不是 `_main`——链接脚本指定入口点为 `_start`，直接在这里开始执行。

- `.section .rodata` 把只读数据明确放在只读段里。

整个程序唯一真正”执行”的部分就是打印——它不会从用户键盘读，也不会做更复杂的事。后面你想扩展它的话，就是在 `halt` 之前加更多功能（比如轮询 UART 的接收寄存器 `[x0, #0x00]` 从用户键盘读）。

3.2 run.sh: 交叉编译与启动脚本

3.2.1 为什么需要三步式构建

在 macOS 上写一个在 QEMU 内跑的 ARM64 裸机程序，你面对的是”跨平台”问题：你的电脑是 ARM64 架构，但你写的程序跑在另一个”ARM64 机器”里。实际上，它们指令集是一样的，区别只在地址空间和系统调用约定上。

具体来说：

- macOS 的 AArch64 用户态程序通过 `svc #0x80` 调用内核
- 裸机 AArch64 程序根本没有内核，也没有 `svc` 约定
- 因此我们需要一个”不链接任何系统库”的纯裸机二进制

因此构建是三步：

```

1 #!/bin/zsh
2 set -e
3 mkdir -p bin
4
5 # 第一步：用 clang 作为交叉编译器
6 # --target=aarch64-none-elf 告诉 clang 目标是裸机 ARM64
7 # -c 只编译不链接，产出 bin/start.o
8 /opt/homebrew/opt/llvm/bin/clang \
9     --target=aarch64-none-elf \
10    -c start.s \
11    -o bin/start.o
12
13 # 第二步：用 LLVM 的 ld.lld 链接
14 # -Ttext=0x40000000 告诉链接器把 .text 段放到 0x4000\_0000
15 # 这是 QEMU virt 机器的内核默认加载地址
16 # -e _start 指定入口符号
17 /opt/homebrew/opt/llvm/bin/ld.lld \
18     -Ttext=0x40000000 \
19     -e _start \
20     -o bin/kernel.elf bin/start.o
21
22 # 第三步：启动 QEMU
23 # -machine virt : 使用 virt 虚拟机器（通用 ARM64 虚拟平台）

```

```
24 # -cpu cortex-a72 : 模拟 ARM Cortex-A72 处理器
25 # -nographic : 用当前终端作为串口控制台（不弹出图形窗口）
26 # -kernel bin/kernel.elf : 把 ELF 文件作为内核加载
27 qemu-system-aarch64 \
28     -machine virt \
29     -cpu cortex-a72 \
30     -nographic \
31     -kernel bin/kernel.elf
```

Listing 3.2: QEMU/run.sh —编译脚本

地址 0x4000_0000 的来源 QEMU 的 virt 机器把”内核可加载”的 RAM 起始地址固定为 0x4000_0000（1 GB 处）。你编译时让链接器把 `_start` 放到这个地址，然后 QEMU 从这个地址开始执行你的程序。

如果你想让程序执行在别的地址，可以随意改这个数字，但必须保证它在 QEMU 的 DRAM 范围内（0x4000_0000 到 0x8000_0000 之间是一个安全的默认区）。

退出 QEMU 在 `-nographic` 模式下，**Ctrl-A X**（先按 Ctrl-A，松开，再按 X）可以退出 QEMU。Ctrl-C 是被送进模拟机器里的，不会让你退出 QEMU 主程序。

3.2.2 常见错误

- **command not found: ld.lld**: 你的 Homebrew 没装 lld。用 `brew install lld` 或直接 `brew install llvm` 装完整 LLVM。
- **No 'kernel' option found for this machine type**: QEMU 版本太老或你没装 AArch64 版的 QEMU。确认你装的是 `qemu-system-aarch64`。
- **程序跑了但没输出**: 检查 UART 基地址是否为 0x09000000。不同机器类型使用不同地址（例如 `versatilepb` 用的是 0x101f1000）。
- **地址异常或崩溃**: 链接地址与 QEMU 加载地址不一致。用 `-Ttext=0x40000000` 配合 `-kernel bin/kernel.elf`，会按链接地址原样加载各段到对应物理地址。

运行脚本后，终端会输出：

```
Hello QEMU AArch64 bare metal!
```

然后程序进入 `wfi` 永久等待。按 Ctrl-A X 退出 QEMU。

3.3 理解地址空间与 UART

3.3.1 裸机地址空间的样子

QEMU 的 virt 机器把物理内存和外设都映射到同一个地址空间里。CPU 可以通过普通的 `ldr/str` 指令直接访问它们。这叫**内存映射 I/O (MMIO)**。

地址范围	内容
0x0000_0000 – 0x0800_0000	Flash / 只读区 (QEMU 放了固件)
0x0900_0000 – 0x0900_0FFF	UART0 串口 (我们使用这个)
0x0901_0000 – 0x0901_0FFF	UART1 串口
0x4000_0000 – 0x8000_0000	DRAM (1 GB) : 程序放在这里
0x0800_0000 – 0x0800_0FFF	GIC (中断控制器)

所以当你执行 `str w2, [0x0900_0000]` 时, CPU 发出的这个写请求落在 DRAM 之外——它被路由到 UART0 设备, 而不是写进内存。硬件 (QEMU 模拟的硬件) 截获这个地址, 把 `w2` 的值当作一个要发送的字节。

同样地, 执行 `ldr w3, [0x0900_0018]` 时, CPU 从 UART 的 FLAG 寄存器读取当前状态位。

3.3.2 PL011 UART 的寄存器表 (关键部分)

ARM PL011 是一个标准的串口控制器, QEMU 精确模拟了它。每一个寄存器都是 32 位 (4 字节) 宽:

偏移	名称	说明
0x00	UARTDR (数据寄存器)	写 = 发送一个字节; 读 = 读最近收到的字节
0x18	UARTFR (标志寄存器)	bit 4 = RXFE (接收空); bit 5 = TXFF (发送满)

其他寄存器 (波特率、线路控制等) 在 QEMU 的 PL011 里通常已经被模拟层初始化为正常可用的状态, 所以最小程序不必设置它们。你只要能向 DR 发送字节、检查 FR 是否忙碌即可。

下面我们逐行解释 `start.s` 中最关键的两条指令:

读标志寄存器—`ldr w3, [x0, #0x18]` `x0` 指向 0x09000000, 加偏移 0x18 后得到 FLAG 寄存器地址。第 5 位是“发送 FIFO 满”标志, 若为 1 就必须等待。

`tbz w3, #5, wait_uart` —忙等直到不忙 Test Bit and Branch if Not Zero: 如果 `w3` 的第 5 位是 1, 就跳回 `wait_uart` 标签继续读。否则 (发送器空闲), 继续执行。这是硬件层面的“忙等”循环——真实硬件里这个过程可能需要几十纳秒; QEMU 里几乎瞬间完成。

str w2, [x0] —发送字节 把 w2 的内容写入 0x09000000 (UARTDR)。QEMU 看到这个地址被写入，就把低 8 位当作一个 ASCII 字符送到终端。

3.3.3 如果要从用户键盘接收数据

发送的反方向操作是：

1. 读 UARTFR (0x09000018)，检查第 4 位 (RXFE = Receiver FIFO Empty)
2. 如果 RXFE=0，说明接收到了新字节
3. 读 UARTDR (0x09000000)，取出低 8 位，就是用户敲的字符

这对应第 4 章 Verilog 里的 UART 接收器做的事——它们本质是同一套硬件逻辑，一个由你用 Verilog 设计，一个由 QEMU 用 C 模拟。

3.3.4 内存与硬件设备的区别

理解这个模型最重要的一点是：

地址本身没有意义。把地址映射到 RAM 还是设备，由硬件（或 QEMU）决定。

- 写 0x4000_0000 —写到 DRAM，下次读会读到你写的值。
- 写 0x0900_0000 —写到 UART，下次读这个地址也许读到的是一个刚接收到的字节。
- 读 0x0900_0018 —读到的不是 RAM，是硬件状态寄存器。

在真实芯片里，这是由 SOC (System on Chip) 内部的**地址解码器**实现的——它根据地址范围把读写请求路由到不同的模块。你在第 4 章里做的 Verilog UART，就是这个模型中的一个模块。

Chapter 4

Verilog 芯片设计

前面我们在用户态和裸机环境下写软件——我们一直假定 CPU、UART、RAM 这些东西已经存在。本章我们反过来：用 Verilog 写一个简单的硬件模块，让自己理解 CPU 之外的世界是如何工作的。

我们写三个模块：

- **adder.v**: 一个 64 位字节加法器，演示如何用寄存器做简单计算
- **uart_tx.v**: UART 发送器，把并行字节转成串行比特流送出
- **uart_rx.v**: UART 接收器，把外部输入的串行比特流转回并行字节

4.1 Verilog 基本概念

4.1.1 软件和硬件的根本区别

汇编语言是给 CPU 看的——它描述的是“一步一步做什么”。Verilog 是给综合器看的——它描述的是“硬件模块之间如何连接、每个时钟沿做什么”。

并行 vs 顺序 在 C/汇编里，代码是顺序执行的。Verilog 不一样：每个 **always** 块都在同时运行。CPU 内部有几十上百个这样的并行块。

两个基本类型

- **reg**: 寄存器，在时钟沿保存值
- **wire**: 连线，只是把一个模块的输出连到另一个模块的输入

4.1.2 同步逻辑：时钟驱动一切

几乎所有现代数字设计都使用“同步时序逻辑”：

```
1 always @(posedge clk or negedge rst_n)
2 begin
3     if (!rst_n)
4         q <= 1'b0;           // 复位
5     else
6         q <= d;             // 每个时钟上升沿，把 d 的值写入 q
7 end
```


- posedge clk: 时钟上升沿
- negedge rst_n: 复位（低电平有效）
- <=: 非阻塞赋值，在组合逻辑里用 =，在时序逻辑里用 <=
- 1'b0: 表示 1 位二进制 0

4.1.3 UART 通信的基本约定

波特率 每秒发送的比特数，常见值如 115200

数据位 一个字节通常 8 位

停止位 1 位或 2 位，标识字节结束

起始位 1 位，先把电平拉低来通知”我要开始发了”

总位数 1 起始 + 8 数据 + 1 停止 = 10 位 / 字节

UART 不附带时钟信号，收发双方必须事先约定好同一个波特率，然后自己在内部用时钟计数来估计每个比特的中心位置。

4.2 adder.v: 64 位字节加法器

4.2.1 设计目标

把 64 位输入（8 个字节）的每个字节提取出来，求和，输出一个 8 位结果。这是一个纯组合逻辑（不需要时钟）+ 一个寄存器输出的简单示例。

4.2.2 源码与讲解

```

1 module adder
2 #(
3     parameter CLK_FRE = 50,          // 时钟频率(MHz)
4     parameter BAUD_RATE = 115200    // 波特率（这里未使用，但保持与其他模块一致）
5 )
6 (
7     input clk,                        // 时钟输入
8     input rst_n,                      // 异步复位，低电平有效
9     input [63:0] in_register,        // 64 位输入（8 个字节）
10    output reg [7:0] out_register    // 8 位输出（求和结果）
11 );
12
13 always @(posedge clk or negedge rst_n)
14 begin
15     if (!rst_n)
16         begin

```

```

17     out_register <= 8'd0;
18 end
19 else
20 begin
21     out_register
22         <= (in_register >> (8*0) & 8'b11111111)
23         + (in_register >> (8*1) & 8'b11111111)
24         + (in_register >> (8*2) & 8'b11111111)
25         + (in_register >> (8*3) & 8'b11111111)
26         + (in_register >> (8*4) & 8'b11111111)
27         + (in_register >> (8*5) & 8'b11111111)
28         + (in_register >> (8*6) & 8'b11111111)
29         + (in_register >> (8*7) & 8'b11111111);
30 end
31 end
32 endmodule

```

Listing 4.1: Chip/src/adder.v

位提取与掩码

- `in_register >> (8*0)` 不移位；`& 8'hFF` 只保留最低字节
- `in_register >> (8*1)` 右移 8 位，保留第二字节
- ...直到 `8*7 =` 第 8 个字节（最高位）

每个字节都是 8 位，8 个这样的数相加，最大可能值是 $8 \times 255 = 2040$ ，需要至少 11 位存。但我们的 `out_register` 只有 8 位——结果会**溢出并截断**，只保留低 8 位。这是故意的——它演示了”硬件里做的运算宽度是你事先规定好的，超出部分就丢掉”的特点。

综合后会是什么样的硬件 综合器会把这个设计变成一个由 7 个 8 位加法器串起来的电路（加法树），最后一级的和被一个 8 位寄存器锁住。每个时钟沿，寄存器都会重新计算一次——如果输入不变，输出也不变。

如果你想保留完整的 11 位结果，只需把 `out_register` 改成 `[10:0]`（11 位）。其余代码无需改动。

4.2.3 一个更简洁的写法

其实 Verilog 支持用 `for` 循环写重复逻辑：

```

1 // generate 块在综合时被展开成具体的门电路
2 generate
3     for (i = 0; i < 8; i = i + 1)
4         begin : gen_bytes

```

```

5      // 编译时展开：8 个独立的字节提取逻辑
6      end
7  endgenerate

```

Listing 4.2: 用 generate 写的简化版本

但对我们的目标而言——理解硬件在做什么——直接写 8 次反而更清晰，因为你能看到每个字节在做同一件事。

4.3 uart_tx.v: UART 发送器

4.3.1 设计目标

接收一个并行的 8 位字节 (tx_data)，在 tx_data_valid = 1 时捕获它，然后按 UART 协议逐位发送到 tx_pin 上。

协议：

一帧 11 位（起始位 + 8 数据位 + 停止位 + 空闲）：

位	0	1-8	9	10
作用	起始	数据 b0-b7	停止	空闲
电平	低	(数据)	高	高

每一位的持续时间由波特率决定。在 50 MHz 时钟、115200 波特率下，一位持续 $\frac{50,000,000}{115200} \approx 434$ 个时钟周期。

4.3.2 状态机设计

发送器使用 3 个状态的简单状态机：

S_IDLE	空闲。等待 tx_data_valid。
S_START	发送起始位（拉低 tx_pin 一位的时间）。
S_SEND_BYTE	逐位发送 8 个数据位，低位在前。
S_STOP	发送停止位（拉高 tx_pin 一位的时间），完成后回到 IDLE。

4.3.3 源码与讲解

```

1 module uart_tx
2 # (
3     parameter CLK_FRE = 50,          // 时钟频率 (MHz)
4     parameter BAUD_RATE = 115200    // 波特率
5 )
6 (
7     input clk,
8     input rst_n,

```

```

9     input [7:0] tx_data,           // 要发送的字节
10    input tx_data_valid,           // 数据有效：一次握手的开始
11    output reg tx_data_ready,      // 告诉发送方：我已准备好接收下一个
12    output tx_pin                  // 串行输出引脚
13 );
14
15 localparam CYCLE = CLK_FRE * 1000000 / BAUD_RATE;
16 // CYCLE = 434 (50 MHz / 115200)
17
18 // 状态编码
19 localparam S_IDLE      = 1;
20 localparam S_START     = 2;
21 localparam S_SEND_BYTE = 3;
22 localparam S_STOP      = 4;
23
24 reg [2:0] state;
25 reg [15:0] cycle_cnt; // 每一位内的时钟计数 (最多 65535)
26 reg [2:0] bit_cnt;    // 已经发送了第几位 (0..7)
27 reg [7:0] tx_data_latch; // 锁存住要发送的字节
28 reg      tx_reg;       // 输出寄存器
29
30 assign tx_pin = tx_reg; // tx_reg 的值直接送到 tx 引脚

```

Listing 4.3: Chip/src/uart_tx.v

状态转移逻辑:

```

1 always @(posedge clk or negedge rst_n)
2 begin
3     if (!rst_n)
4         state <= S_IDLE;
5     else
6         case (state)
7             S_IDLE:
8                 if (tx_data_valid)
9                     state <= S_START;
10                else
11                    state <= S_IDLE;
12            S_START:
13                if (cycle_cnt == CYCLE - 1)
14                    state <= S_SEND_BYTE;
15                else
16                    state <= S_START;
17            S_SEND_BYTE:
18                if (cycle_cnt == CYCLE - 1 && bit_cnt == 3'd7)
19                    state <= S_STOP;
20                else
21                    state <= S_SEND_BYTE;

```

```

22         S_STOP:
23             if (cycle_cnt == CYCLE - 1)
24                 state <= S_IDLE;
25             else
26                 state <= S_STOP;
27         endcase
28 end

```

数据准备好标志和锁存:

```

1 always @(posedge clk or negedge rst_n)
2 begin
3     if (!rst_n)
4         tx_data_ready <= 1'b0;
5     else if (state == S_IDLE)
6         tx_data_ready <= !(tx_data_valid); // 空闲时才能接收
7     else if (state == S_STOP && cycle_cnt == CYCLE - 1)
8         tx_data_ready <= 1'b1;
9 end
10
11 always @(posedge clk or negedge rst_n)
12 begin
13     if (!rst_n)
14         tx_data_latch <= 8'd0;
15     else if (state == S_IDLE && tx_data_valid)
16         tx_data_latch <= tx_data; // 在进入 START 状态时锁存数据
17 end

```

计数器 (计算当前位已发送了多少个时钟周期):

```

1 always @(posedge clk or negedge rst_n)
2 begin
3     if (!rst_n)
4         cycle_cnt <= 16'd0;
5     else if ((state == S_START || state == S_SEND_BYTE ||
6             state == S_STOP) && cycle_cnt < CYCLE - 1)
7         cycle_cnt <= cycle_cnt + 16'd1;
8     else
9         cycle_cnt <= 16'd0;
10 end
11
12 always @(posedge clk or negedge rst_n)
13 begin
14     if (!rst_n)
15         bit_cnt <= 3'd0;
16     else if (state == S_SEND_BYTE)
17         if (cycle_cnt == CYCLE - 1)
18             bit_cnt <= bit_cnt + 3'd1;
19     else

```

```

20         bit_cnt <= bit_cnt;
21     else
22         bit_cnt <= 3'd0;
23 end

```

最后，真正驱动输出引脚的逻辑：

```

1  always @(posedge clk or negedge rst_n)
2  begin
3      if (!rst_n)
4          tx_reg <= 1'b1;    // UART 空闲时 tx_pin 为高电平
5      else
6          case (state)
7              S_IDLE:      tx_reg <= 1'b1;    // 空闲=高
8              S_START:    tx_reg <= 1'b0;    // 起始位=低
9              S_SEND_BYTE:
10                 tx_reg <= tx_data_latch[bit_cnt]; // 从 bit0 开始逐位发
11              S_STOP:     tx_reg <= 1'b1;    // 停止位=高
12              default:    tx_reg <= 1'b1;
13          endcase
14  end
15 endmodule

```

锁存为什么重要 tx_data_latch 在进入 S_START 时把 tx_data 的值存下来。如果没有这一步，上游逻辑在发送过程中改变 tx_data，我们会输出一个被污染的字节——先几位是旧值，后几位是新值。

为什么 UART 空闲时 tx_pin 必须为高 UART 协议里，“线路空闲”就是高电平。起始位是把线路拉低一位时间，这样接收端才能检测到“有数据开始到来”。如果空闲时电平是低，接收端会把空闲状态误当做起始位，整个通信就错乱了。

4.4 uart_rx.v: UART 接收器

4.4.1 设计目标

与发送器相反：持续监视 rx_pin。当 rx_pin 从 1 变 0（起始位到来）时，启动一个计数器。在每个比特的中心位置采样一次电平，攒够 8 位后输出 rx_data 并拉高 rx_data_valid。

接收器比发送器难，因为：

- 发送器是自己决定什么时候发一位，时钟完全可控
- 接收器不知道对方何时开头发送，必须自己检测起始位

- 接收器不能保证自己的时钟和对方时钟完全对齐，必须在比特**中心**采样以获得最大噪声容限

4.4.2 状态机设计

S_IDLE	监视 rx_pin。检测到下降沿 → 进入 START 状态。
S_START	计数半个周期，再采样一次确认真的是低电平（防毛刺）。
S_REC_BYTE	每过一个周期采样一位，共 8 位，低位在前。
S_STOP	等待停止位，完成后回到 IDLE。
S_DATA	拉高 valid 标志，让上层模块取走字节。

4.4.3 源码与关键思路讲解

```

1 module uart_rx
2 #(
3     parameter CLK_FRE = 50,
4     parameter BAUD_RATE = 115200
5 )
6 (
7     input clk,
8     input rst_n,
9     output reg [7:0] rx_data,          // 收到的字节
10    output reg rx_data_valid,          // 一次有效，表明有新字节
11    input rx_data_ready,               // 上层已取走，让我进入下一轮
12    input rx_pin                       // 串行输入引脚
13 );
14
15 localparam CYCLE = CLK_FRE * 1000000 / BAUD_RATE;
16 localparam S_IDLE      = 1;
17 localparam S_START     = 2;
18 localparam S_REC_BYTE  = 3;
19 localparam S_STOP      = 4;
20 localparam S_DATA      = 5;
21
22 reg [2:0] state;
23 reg [2:0] next_state;
24 reg      rx_d0;          // 打一拍，同步 rx_pin
25 reg      rx_d1;          // 再打一拍，用于检测下降沿
26 wire     rx_negedge;     // 检测到一个下降沿
27 reg [7:0] rx_bits;        // 保存逐位收到的数据
28 reg [15:0] cycle_cnt;     // 当前位已持续了多久
29 reg [2:0] bit_cnt;        // 已收到第几位

```

Listing 4.4: Chip/src/uart_rx.v

打两拍：跨时钟域的基本操作 rx_pin 是一个从外部引脚进入的信号，它的变化不一定对齐我们的时钟沿。直接用它来驱动逻辑会有“亚稳态”风险。解决办法是**打两拍**：

```

1 assign rx_negedge = rx_d1 && ~rx_d0; // 检测到从 1→0
2
3 always @(posedge clk or negedge rst_n)
4 begin
5     if (!rst_n)
6     begin
7         rx_d0 <= 1'b0;
8         rx_d1 <= 1'b0;
9     end
10    else
11    begin
12        rx_d0 <= rx_pin;
13        rx_d1 <= rx_d0;
14    end
15 end

```

打两拍后，我们内部观察到的信号变化至少延迟了两个时钟周期，但它是**稳定的**——不会在一个时钟沿前后出现混乱的值。

状态转移逻辑：使用两个 always 块 接收器用了“状态 / 次态分离”的写法，这是更标准的状态机写法：

```

1 // 第一块：状态更新（时序逻辑，只在 clk 沿更新）
2 always @(posedge clk or negedge rst_n)
3 begin
4     if (!rst_n)
5         state <= S_IDLE;
6     else
7         state <= next_state;
8 end
9
10 // 第二块：计算 next_state（组合逻辑，异步计算）
11 always @(*)
12 begin
13     case (state)
14         S_IDLE:
15             if (rx_negedge)
16                 next_state = S_START;
17             else
18                 next_state = S_IDLE;
19         S_START:
20             if (cycle_cnt == CYCLE - 1)
21                 next_state = S_REC_BYTE;
22             else
23                 next_state = S_START;

```



```

24     S_REC_BYTE:
25         if (cycle_cnt == CYCLE - 1 && bit_cnt == 3'd7)
26             next_state = S_STOP;
27         else
28             next_state = S_REC_BYTE;
29     S_STOP:
30         if (cycle_cnt == CYCLE/2 - 1) // 半位就进入数据锁存
31             next_state = S_DATA;
32         else
33             next_state = S_STOP;
34     S_DATA:
35         if (rx_data_ready)
36             next_state = S_IDLE;
37         else
38             next_state = S_DATA;
39     default:
40         next_state = S_IDLE;
41 endcase
42 end

```

为什么在 S_STOP 里只等半位 我们是在起始位的中间检测到稳定低电平。后续每位也都是从位中心开始的一个完整周期采样。但停止位其实不需要等完整的一位——只要确认它是高电平就完成了。所以用 $CYCLE/2 - 1$ ，只等半个周期，省出的时间让我们更快回到 IDLE 状态，不漏掉下一个字节。

数据与有效标志

```

1 always @(posedge clk or negedge rst_n)
2 begin
3     if (!rst_n)
4         rx_data_valid <= 1'b0;
5     else if (state == S_STOP && next_state != state)
6         rx_data_valid <= 1'b1;
7     else if (state == S_DATA && rx_data_ready)
8         rx_data_valid <= 1'b0;
9 end
10
11 always @(posedge clk or negedge rst_n)
12 begin
13     if (!rst_n)
14         rx_data <= 8'd0;
15     else if (state == S_STOP && next_state != state)
16         rx_data <= rx_bits; // 把 8 位并行输出
17 end

```

逐位接收

```

1 always @(posedge clk or negedge rst_n)
2 begin
3     if (!rst_n)
4         cycle_cnt <= 16'd0;
5     else if ((state == S_START || state == S_REC_BYTE ||
6             state == S_STOP) && cycle_cnt < CYCLE - 1)
7         cycle_cnt <= cycle_cnt + 16'd1;
8     else
9         cycle_cnt <= 16'd0;
10 end
11
12 always @(posedge clk or negedge rst_n)
13 begin
14     if (!rst_n)
15         rx_bits <= 8'd0;
16     else if (state == S_REC_BYTE && cycle_cnt == CYCLE/2 - 1)
17         rx_bits[bit_cnt] <= rx_pin;    // 在每个比特的中心采样
18 end
19
20 endmodule

```

中心采样的重要性 在 $\text{cycle_cnt} == \text{CYCLE}/2 - 1$ 时采样，相当于在每个位的正中间读取电平。UART 通信两端的时钟总有微小的误差，但只要误差不累计到超过半个比特周期，中心采样就可以正确读取数据。这是 UART 协议的核心鲁棒性来源。

如果在边沿采样，发送器和接收器的时钟漂移累积起来，几个字节之后你就会开始读错——因为你采样的位置在两个比特的交界处，读出的电平取决于先到哪一侧。

4.5 综合到 FPGA：从代码到硬件

4.5.1 从 Verilog 到真实硬件的流程

写好的 Verilog 代码通过下面几个步骤变成可以烧到 FPGA 里的文件：

1. **综合 (Synthesis)**：把 Verilog 代码翻译成一个由逻辑门、触发器、查找表 (LUT) 组成的网表
2. **布局 (Placement)**：决定每个逻辑门放在 FPGA 的哪个位置
3. **布线 (Routing)**：在 FPGA 的可编程互连资源里把需要通信的逻辑门连接起来
4. **时序分析 (Timing Analysis)**：验证设计能在目标时钟频率下正确工作——每条路径的传播延迟必须小于一个时钟周期

5. **生成比特流 (Bitstream Generation)**: 把最终配置信息打包成 FPGA 芯片可以理解的二进制文件

你手工用的工具通常是 Vivado (Xilinx 系列 FPGA)、Quartus (Intel/Altera) 或者开源工具链 (Yosys + nextpnr + icestorm, 针对 Lattice FPGA)。

4.5.2 u_lite 程序如何加载到 FPGA 的指令存储器

在设计一个自制 CPU 时, 指令内存有两种主要的初始化方式:

方式 1: 利用 Block RAM 初始化 在综合时, 把你汇编产出的机器码写成一个初始化文件 (.coe 或 .mem), 让综合器在编译时把这些值写进 FPGA 的 BRAM 里。

```

1 // 这个模块在综合时被工具识别为 "Single Port RAM with initial contents"
2 reg [31:0] instr_mem [0:1023]; // 1024 条 32 位指令 = 4 KB
3
4 initial begin
5     $readmemh("code.mem", instr_mem); // 综合时被解析
6 end
7
8 // 每个时钟沿读出一条指令
9 always @(posedge clk)
10 begin
11     instr_out <= instr_mem[pc[11:2]];
12 end

```

Listing 4.5: BRAM 里嵌入机器码的模板

在编译(汇编)层面, 你把 start.s 这类文件用 clang --target=aarch64-none-elf -c start.s -o start.o 产出.o, 再用 llvm-objcopy -O binary 产出纯二进制, 最后写一段脚本把它转成一行一个十六进制数的.mem 文件。

方式 2: 通过 UART bootloader 更灵活的做法: 在 FPGA 设计里放一个小的 bootloader (固定在 BRAM 里), 它在上电后立即启动, 通过 UART 接收外部发来的程序二进制, 把它写入一块更大的 RAM/ROM, 然后跳过去执行。

这对应 QEMU 章节里你所看到的思路: 程序可以在运行时被写入硬件。UART 发送器 (uart_tx) 把程序字节送来, UART 接收器 (uart_rx) 把它们读出来, 写进指令存储器——你就不需要重新综合了。

4.5.3 和 QEMU 模拟的对应关系

现在把第 3 章的 QEMU 裸机程序和本章的 Verilog UART 做个对比:

层面	QEMU 里	FPGA 里对应的
UART 设备	虚拟 PL011	你写的 UART 模块
地址	内存路由到 PL011	地址解码器路由到 UART
CPU 取指	TCG 翻译执行 ARM	你未来写的 CPU 核
DRAM	QEMU 模拟的内存	FPGA 片上 BRAM 或 DDR

你现在写的 UART 模块，加上一个简单的 CPU，就是一个最小的 SoC。把之前写的 QEMU 汇编程序（读取输入）改写成这个自制 CPU 可以执行的指令格式后，就能在 FPGA 里跑起来。

4.5.4 为什么要做这种事

从学习角度，写过 UART 之后再回头看 QEMU 里的那段汇编：

```
1 wait_uart:
2     ldr  w3, [x0, #0x18]
3     tbnz w3, #5, wait_uart
4     str  w2, [x0]
```

你会立刻理解它的意思——这是在读取 FLAG 寄存器、检查发送满、然后写数据寄存器。这些动作不是某个抽象的”系统调用”，而是真正把字节送到一个你自己用 Verilog 写过的硬件模块里。你知道电路里发生了什么，就知道软件层为什么要这么写。

反过来说，真正写过软件之后再去写硬件，你也清楚 UART 的数据寄存器和状态寄存器为什么要分开成两个地址——因为软件会分开读它们。两者是一体两面。

Chapter 5

C 语言算法与数据结构

对应项目目录：algo/src/。

5.1 本章讲什么

本章用 C 语言实现了一组经典算法。我们关注：

- **插入排序**—最朴素的排序思路，理解「逐个插入」
- **归并排序（含逆序对统计）**—分治思想的典范
- **堆排序**—把数组看成二叉树，原地 $O(n \log n)$
- **最大子数组问题**—暴力枚举 vs 分治法
- **矩阵乘法（分治法）**—二维数据的递归拆分

每一节都包含你自己写的 C 源码，并解释核心逻辑。

5.2 插入排序—逐个插入

插入排序（Insertion Sort）是最直观的排序算法之一。它的工作方式很像你一手拿一叠扑克牌，另一只手把每一张牌插入到合适的位置。

5.2.1 核心思路

从第二个元素开始遍历数组。对于每个元素 $A[j]$ ：

1. 把它暂存为 **key**
2. 把 **key** 左边所有比它大的元素整体右移一格
3. 最后把 **key** 放到腾出来的位置上

这样，每处理完一个元素，它左边的部分就是有序的。

5.2.2 时间复杂度

最好情况	数组已经有序	$O(n)$
最坏情况	数组完全逆序	$O(n^2)$
平均情况	随机数据	$O(n^2)$

对小规模数据或几乎有序的数据来说，插入排序实际上很快，因为它空间开销为 $O(1)$ 且常数因子很小。

5.2.3 C 代码

```

1 // 插入排序
2 #include <stdio.h>
3 void insertion_sort(int A[],int n){
4     for (int j=1;j<n;j++){
5         {
6             int key=A[j]; // 选择当前元素作为关键值
7             int i=j-1; // 从当前元素的前一个元素开始比较
8             while (i>=0 && A[i]>key)
9                 {
10                     A[i+1]=A[i];///如果当前元素大于关键值，将当前元素向后移动
11                     i=i-1;/// 继续比较前一个元素
12                 }
13             A[i+1]=key;/// 将关键值插入到正确位置
14         }
15     }
16 int main(){
17     int A[] = {5,2,4,6,1,3,7,8};
18     int n = sizeof(A)/sizeof(A[0]); // 计算数组元素个数
19     insertion_sort(A,n);
20     for(int i=0;i<n;i++)
21     {
22         printf("%d ",A[i]);// 输出排序后的数组元素
23     }
24     printf("\n");
25     return 0;
26 }
```

Listing 5.1: algo/src/sort/inserton_sort.c —插入排序

5.2.4 代码解读

这段代码的骨架非常简洁：

```

1 void insertion_sort(int A[], int n) {
2     for (int j = 1; j < n; j++) {        // 从第二个元素开始
```

```

3     int key = A[j];           // 拿出当前元素
4     int i = j - 1;           // i 指向已排序部分末尾
5     while (i >= 0 && A[i] > key) { // 向左找插入点
6         A[i + 1] = A[i];      // 元素右移腾出空间
7         i = i - 1;
8     }
9     A[i + 1] = key;          // 放入正确位置
10 }
11 }

```

几个值得注意的点：

- $A[i] > \text{key}$ 决定了排序是升序的；如果改成 $A[i] < \text{key}$ ，就变成降序。
- 内层 `while` 循环的「边比较边后移」是典型的原地插入方式，不需要额外空间。
- 整个算法是**稳定的**：相等元素不会互相交换相对位置。

编译运行：

```

1 clang -o insert algo/src/sort/inserton_sort.c
2 ./insert
3 2 4 5 6 1 3 7 8

```

5.3 归并排序—分治 + 逆序对

归并排序（Merge Sort）是「分治」思想最经典的应用之一。把一个大数组切成两半，分别排序，再把两个有序数组合并成一个。

5.3.1 核心思路

分治的三个阶段：

1. **分解（Divide）**：把数组从中间切开，得到两个子数组
2. **解决（Conquer）**：递归地对两个子数组调用归并排序
3. **合并（Merge）**：把两个有序的子数组合并成一个有序数组

5.3.2 时间复杂度

因为每次都把数组对半切开，递归树的深度是 $O(\log n)$ ，每一层合并的总工作量是 $O(n)$ ，所以总时间 $O(n \log n)$ 。代价是需要一块大小为 $O(n)$ 的辅助数组 `B[]`。

5.3.3 C 代码—合并函数

核心在于 Merge：把 $A[\text{low}..\text{mid}]$ 和 $A[\text{mid}+1..\text{high}]$ 这两段已有序的部分合并为 $A[\text{low}..\text{high}]$ 。

```

1 #include <stdio.h>
2
3 int count; //逆序对个数
4 void Merge(int low, int mid, int high, int A[], int B[])
5 {
6     int i, j, k;
7     for(k=low; k<=high; k++)
8     {
9         B[k]=A[k];
10    }
11    i=low;
12    j=mid+1;
13    k=low;
14    while(i<=mid && j<=high)
15    {
16        if(B[i]<B[j])
17        {
18            A[k++]=B[i++];
19        }
20        else
21        {
22            A[k++]=B[j++];
23            count+=mid-i+1;
24        }
25    }
26    while(i<=mid)
27    {
28        A[k++]=B[i++];
29    }
30    while(j<=high)
31    {
32        A[k++]=B[j++];
33    }
34 }
```

Listing 5.2: src/sort/merge_sort.c —Merge 函数（含逆序对计数）

这段代码还顺便统计了**逆序对**（inversion）——也就是数组中‘ $i < j$ 但 $A[i] > A[j]$ ’的情况数。关键一行是：

```

1 count += mid - i + 1; // 当从右半部分取出元素时
```

意思是：当 $B[j]$ 比 $B[i]$ 小，说明从 $B[i]$ 到 $B[\text{mid}]$ 这些元素全都是比 $B[j]$ 大的，每一个都和 $B[j]$ 构成一个逆序对。

5.3.4 C 代码—递归排序函数与主程序

```

1 void Mergesort(int low,int high,int A[],int B[])
2 {
3     if(low<high)
4     {
5         int mid=(low+high)/2;
6         Mergesort(low,mid,A,B);
7         Mergesort(mid+1,high,A,B);
8         Merge(low,mid,high,A,B);
9     }
10 }
11
12 int main()
13 {
14     count=0;
15     int A[] = {0,5,2,4,6,1,3,7,8};
16     int n = sizeof(A)/sizeof(A[0]) - 1;
17     int B[n+1];
18     Mergesort(1,n,A,B);
19     for(int i=1;i<=n;i++)
20     {
21         printf("%d ",A[i]);
22     }
23     printf("\n");
24     printf("逆序对个数为: %d\n",count);
25     return 0;
26 }

```

Listing 5.3: src/sort/merge_sort.c —Mergesort 递归函数 + main

注意这段代码使用了**从 1 开始**的数组下标习惯 ($A[1..n]$)，这是《算法导论》里常用的写法，便于计算 mid 。如果换成从 0 开始，只要把 low 的初始值改成 0、 $high$ 改成 $n-1$ 即可。

5.3.5 编译与运行

```

1 clang -o merge algo/src/sort/merge_sort.c
2 ./merge
3 1 2 3 4 5 6 7 8
4 逆序对个数为: 10

```

对于测试数组 $\{0, 5, 2, 4, 6, 1, 3, 7, 8\}$ （位置 0 被忽略），逆序对共 10 对：(5,2), (5,1), (5,3), (2,1), (4,1), (4,3), (6,1), (6,3), 等等。这给了你一个判断排序实现是否正确的额外指标。

5.4 堆排序—把数组当二叉树

堆排序 (Heapsort) 把数组想象成一棵二叉树——这就是「堆」这个名字的来源。它不需要额外的数据结构，只在原数组上通过下标关系操作。

5.4.1 核心思路：把数组看成一棵二叉树

堆排序用一种巧妙的下标关系：

- 节点 i 的父节点是 $i/2$
- 节点 i 的左孩子是 $2*i$
- 节点 i 的右孩子是 $2*i+1$

这个映射不需要额外内存，只靠四则运算。注意它沿用了《算法导论》的从 1 开始的数组习惯—— $A[0]$ 通常放一个哨兵或被忽略。

一个**最大堆**满足：每个父节点 $\geq$ 它的两个孩子，即 $A[\text{Parent}(i)] \geq A[i]$ 对所有 i 成立。这保证了根节点 $A[1]$ 是整个堆中的最大值。

5.4.2 C 代码—堆性质维护 Maxheapify

排序的核心是一个叫 Maxheapify 的函数：当下述三个节点中有节点违反最大堆性质时，把最大的元素换到父节点位置，然后递归下去。

```
1 void Maxheapify(int A[],int heap_size,int i)
2 {
3     int l=Leftchild(i);
4     int r=Rightchild(i);
5     int largest=i;
6     if(l<=heap_size && A[l]>A[largest])
7     {
8         largest=l;
9     }
10    if(r<=heap_size && A[r]>A[largest])
11    {
12        largest=r;
13    }
14    if(i!=largest)
15    {
16        int temp=A[i];
17        A[i]=A[largest];
18        A[largest]=temp;
19        Maxheapify(A,heap_size,largest);
20    }
21 }
```

Listing 5.4: algo/src/sort/heap_sort.c —Maxheapify

它的工作方式可以概括为：

1. 看当前节点 i 和它两个孩子里谁最大
2. 如果当前节点已经最大，什么也不做
3. 否则把当前节点和最大的那个孩子交换
4. 到被交换的那一侧递归调用 Maxheapify

因为每次最多下一层，所以一棵有 n 个节点的树，深度最多 $O(\log n)$ ，每次调用 Maxheapify 的时间为 $O(\log n)$ 。

5.4.3 C 代码—建堆 BuildmaxHeap

建堆从最后一个非叶节点开始（也就是 $\text{Parent}(n)$ ），从后往前对每个节点调用 Maxheapify。这样保证在处理第 i 层时， $i+1$ 层已经是堆。

```
1 void BuildmaxHeap(int A[],int n)
2 {
3     int heap_size=n;
4     for(int i=Parent(n);i>=1;i--)
5     {
6         Maxheapify(A,heap_size,i);
7     }
8 }
```

Listing 5.5: algo/src/sort/heap_sort.c —BuildmaxHeap

为什么从 $\text{Parent}(n)$ 开始而不是 n ？因为超过 $\text{Parent}(n)$ 的节点全都是叶子，一个元素的堆本来就满足堆性质，不需要处理。

5.4.4 C 代码—Heapsort 主函数

排序只需要：1. 把原数组建成最大堆 2. 反复把根节点（最大值）换到数组末尾，堆大小减 1，再从根节点维护堆性质

```
1 void Heapsort(int A[],int n)
2 {
3     int heap_size=n;
4     BuildmaxHeap(A,n);
5     for(int i=n;i>=2;i--)
6     {
7         int temp=A[1];
8         A[1]=A[i];
```

```
9     A[i]=temp;
10     heap_size--;
11     Maxheapify(A,heap_size,1);
12 }
```

Listing 5.6: algo/src/sort/heap_sort.c —Heapsort

第 54–55 行的 `temp = A[1]; A[1] = A[i]; A[i] = temp;` 就是把根换到末尾的操作，相当于把当前的最大值放到了它最终的有序位置。

5.4.5 C 代码—辅助函数与测试

三个索引计算函数很简单，但它们让排序逻辑语义清晰：

```
1 #include <stdio.h>
2
3 int Parent(int i)
4 {
5     return i/2;
6 }
7
8 int Leftchild(int i)
9 {
10    return 2*i;
11 }
12
13 int Rightchild(int i)
14 {
15    return 2*i+1;
16 }
```

Listing 5.7: algo/src/sort/heap_sort.c —Parent/Leftchild/Rightchild

5.4.6 C 代码—main 测试程序

```
1 int main()
2 {
3     int A[] = {0,5,2,4,6,1,3,7,8};
4     int n = sizeof(A)/sizeof(A[0]) - 1;
5     Heapsort(A,n);
6     for(int i=1;i<=n;i++)
7     {
8         printf("%d ",A[i]);
9     }
10    printf("\n");
11    return 0;
12 }
```

Listing 5.8: algo/src/sort/heap_sort.c —main 测试

测试数组 {0,5,2,4,6,1,3,7,8} 中 A[0]=0 被忽略（占位），对 A[1..8] 排序。注意 $\text{sizeof}(A)/\text{sizeof}(A[0]) - 1$ 正好跳过了占位元素，得到有效长度 $n=8$ 。

5.4.7 时间复杂度

| 操作 | 一次调用 | 总次数 | |——|———:|——-:| | Maxheapify | $O(\log n)$ | $O(n)$ 次
在建堆阶段 + $O(n)$ 次排序阶段 | | BuildmaxHeap | —| $O(n)$ （紧界，不是 $O(n \log n)$!）
| | Heapsort | —| $O(n \log n)$ |

为什么 BuildmaxHeap 是 $O(n)$ 而不是 $O(n \log n)$? 因为大部分节点在树的底部（叶子附近），Maxheapify 在那些节点几乎立刻返回。精确求和：

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O(n)$$

而排序阶段每次从根维护堆，深度最大，因此是 $O(n \log n)$ 。

5.4.8 与其他排序的对比

算法	最坏时间	原地?
插入排序	$O(n^2)$	是
归并排序	$O(n \log n)$	否（需要辅助数组）
堆排序	$O(n \log n)$	是

堆排序在**最坏情况仍保证** $O(n \log n)$ ，同时只需要 $O(1)$ 额外空间，这在嵌入式/资源受限场景很有用。

5.5 最大子数组——从暴力到分治

最大子数组问题（Maximum Subarray）：给定一个整数数组，找出一段连续子数组，使得它的元素之和最大。这个问题源于股票买卖——把每天的股价差当作数组元素，找到「哪天买、哪天卖」最划算。

你在项目中实现了**两种**解法，很好地展示了从朴素到高效的演进。

5.5.1 解法一：暴力枚举所有可能的子数组

对每一对下标 (i, j) 都计算 $\sum A[i..j]$ ，取最大。共 $O(n^2)$ 个子数组，每个求和 $O(1)$ （因为用 $A[j] - A[i]$ 的差分思路，实际只计算了两端差值）。

```
1 #include <stdio.h>
2
```

```
3 struct Result
4 {
5     int low;
6     int high;
7     int sum;
8 };
9 struct Result findmaximumsubarray(int A[],int n)
10 {
11     struct Result res;
12     int B[n*(n-1)/2];
13     int C[n*(n-1)/2];
14     int D[n*(n-1)/2];
15     int k=0;
16     for(int i=0;i<n;i++)
17     {
18         for(int j=i+1;j<n;j++)
19         {
20             C[k]=j;
21             D[k]=i;
22             B[k++]=A[j]-A[i];
23         }
24     }
25     int max=B[0];
26     int maxindex=0;
27     for (int i = 1; i < n*(n-1)/2; i++)
28     {
29         if (B[i] > max)
30         {
31             max = B[i];
32             maxindex=i;
33         }
34     }
35     res.high=C[maxindex];
36     res.low=D[maxindex];
37     res.sum=max;
38     return res;
39 }
40 int main()
41 {
42     int A[]={100,113,110,85,105,102,86,63,81,101,94,106,101,79,94,90,97};
43     int n=sizeof(A)/sizeof(A[0]);
44     struct Result data=findmaximumsubarray(A,n);
45     int low=data.low;
46     int high=data.high;
47     int sum=data.sum;
48     printf("%d,%d,%d,%d \n",n,low,high,sum);
49     return 0;
```

50 }

Listing 5.9: algo/src/findmaximumsubarray/1.c —暴力枚举法

这段代码的思路：

- 对所有 $i < j$ ，计算 $A[j] - A[i]$ ，代表「从 i 买入、 j 卖出」的收益
- 顺便记录是哪一对 (i, j) 产生了最大收益
- 时间复杂度 $O(n^2)$ ，空间 $O(n)$ 来存所有对

对小规模数据（比如 $n = 17$ ）这样写完全没问题，结果正确，但当 n 达到几万或更大时，二次方的开销就难以接受了。

5.5.2 解法二：分治法

把数组从中间 `mid` 切开。最大子数组必然落在以下三种情况之一：

1. 完全在左半部分 $A[\text{low}..\text{mid}]$
2. 完全在右半部分 $A[\text{mid}+1..\text{high}]$
3. 跨越 `mid`：一部分在左、一部分在右

三种情况都递归处理，最后取三者中最大的。

跨中点的最大子数组—findmaxcrossingsubarray

```
1 #include <stdio.h>
2 struct Result
3 {
4     int low;
5     int high;
6     int sum;
7 };
8 struct Result findmaxcrossingsubarray(int A[], int low, int mid, int high)
9 {
10     int leftsum=-1000000;
11     int rightsum=-1000000;
12     int sum=0;
13     int max_left;
14     int max_right;
15     for(int i=mid;i>=low;i--)
16     {
17         sum+=A[i];
18         if(sum>leftsum)
```

```
19     {
20         leftsum=sum;
21         max_left=i;
22     }
23 }
24 sum=0;
25 for(int i=mid+1;i<=high;i++)
26 {
27     sum+=A[i];
28     if(sum>rightsum)
29     {
30         rightsum=sum;
31         max_right=i;
32     }
33 }
34 return (struct Result){max_left,max_right,leftsum+rightsum};
35 }
```

Listing 5.10: algo/src/findmaximumsubarray/2.c 一跨中点的查找

逻辑：

- 从 mid 向左遍历，累加求和并更新最大值和对应位置 max_left
- 从 mid+1 向右遍历，同样记录 max_right
- 结果是 (max_left, max_right, leftsum + rightsum)
- 这一步是 $O(n)$ 的，因为只从中间向两边各扫描一次

注意你设 leftsum 和 rightsum 的初值为 -1000000。这个哨兵值很重要——因为数组元素可以是负数，必须让首次比较「哪怕是一个负数的子数组」也能被选中。如果设为 0，当所有数都是负数时算法就会返回错误的空数组。

主递归函数

```
1 struct Result findmaximumsubarray(int A[],int low,int high)
2 {
3     if(high==low)
4     {
5         return (struct Result){low,high,A[low]};
6     }
7     else
8     {
9         int mid=(low+high)/2;
10        struct Result resleft=findmaximumsubarray(A,low,mid);
11        struct Result resright=findmaximumsubarray(A,mid+1,high);
```



```

12     struct Result rescross=findmaxcrossingsubarray(A,low,mid,high);
13     if(resleft.sum>=resright.sum && resleft.sum>=rescross.sum)
14     {
15         return resleft;
16     }
17     else if (resright.sum>=resleft.sum && resright.sum>=rescross.sum)
18     {
19         return resright;
20     }
21     else
22     {
23         return rescross;
24     }
25 }
26 }

```

Listing 5.11: algo/src/findmaximumsubarray/2.c —findmaximumsubarray 递归函数

输入预处理和输出结果

```

1 int main()
2 {
3     int B[]={100,113,110,85,105,102,86,63,81,101,94,106,101,79,94,90,97};
4     int n=sizeof(B)/sizeof(B[0]);
5     int A[n-1];
6     for(int i=0;i<n-1;i++)
7     {
8         A[i]=B[i+1]-B[i];
9     }
10    int len=sizeof(A)/sizeof(A[0]);
11    struct Result data=findmaximumsubarray(A,0,len-1);
12    printf("%d,%d,%d,%d \n",len,data.low,data.high,data.sum);
13    return 0;
14 }

```

Listing 5.12: algo/src/findmaximumsubarray/2.c —main, 把股价转成差分数组

5.5.3 时间复杂度对比

解法	时间	空间	备注
暴力（解法一）	$O(n^2)$	$O(n)$	代码最短，适合调试
分治（解法二）	$O(n \log n)$	$O(\log n)$ 递归栈	大规模数据

你在项目中保留了两种实现，正好可以对照运行来验证正确性——这是学习算法的一种很实用的方式：写一个简单但正确的版本，再写一个快的版本，让两者对拍。

5.6 矩阵乘法—二维分治

两个方阵相乘 $C = A \times B$ ，定义为：

$$C[i][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j]$$

直接实现是三重循环 $O(n^3)$ 。分治法的思路是：把每个矩阵切成 2×2 四块，然后对四块分别递归相乘再合并。

5.6.1 数据结构

代码里用了两个自定义类型：

```
1 typedef int** Matrix;           // 二维数组（指针数组）
2
3 typedef struct {
4     Matrix m11, m12, m21, m22;  // 四块子矩阵
5 } SplitMat;
```

每个矩阵 `Matrix` 实际上是一个指针数组，`mat[i]` 指向第 `i` 行的元素。这样写有一个好处：`mat[i][j]` 等价于 `*(mat[i] + j)`，与汇编里计算二维数组元素地址的方式本质一样——先算行偏移，再算列偏移，最后取内存。

5.6.2 C 代码—创建、释放、打印矩阵

```
1 Matrix createMatrix(int size)
2 {
3     Matrix mat=(Matrix)malloc(size*sizeof(int*));
4     for(int i=0;i<size;i++)
5     {
6         mat[i]=(int*)malloc(size*sizeof(int));
7     }
8     return mat;
9 }
10
11 void freeMatrix(Matrix mat, int size) {
12     for (int i = 0; i < size; i++) {
13         free(mat[i]);
14     }
15     free(mat);
16 }
17 void printMatrix(Matrix mat,int size)
18 {
19     for(int i=0;i<size;i++)
20     {
```

```

21     for(int j=0;j<size;j++)
22     {
23         printf("%d ",mat[i][j]);
24     }
25     printf("\n");
26 }
27 }

```

Listing 5.13: algo/src/matrix/1.c —createMatrix / freeMatrix / printMatrix

5.6.3 C 代码—补零到 2 的幂次方

分治法的前提是矩阵边长是 2 的幂，这样每次能严格对半切。对任意大小的矩阵，我们先把它补零到下一个 2 的幂：

```

1 int nextpoweroftwo(int n)
2 {
3     return (int)pow(2,ceil(log2(n)));
4 }
5
6 Matrix padMatrix(Matrix origin,int originsize)
7 {
8     int padsize=nextpoweroftwo(originsize);
9     Matrix pad=createMatrix(padsize);
10    for(int i=0;i<padsize;i++)
11    {
12        for(int j=0;j<padsize;j++)
13        {
14            if(i<originsize && j<originsize)
15            {
16                pad[i][j]=origin[i][j];
17            }
18            else
19            {
20                pad[i][j]=0;
21            }
22        }
23    }
24    return pad;
25 }

```

Listing 5.14: algo/src/matrix/1.c —nextpoweroftwo + padMatrix

注意 `pow(2, ceil(log2(n)))` 这样的浮点写法在实际工程中可能有精度问题，但作为演示足够直观。更严谨的整数实现可以用位运算：

```

1 int nextpoweroftwo(int n) {
2     int p = 1;

```

```

3   while (p < n) p <= 1;
4   return p;
5 }

```

5.6.4 C 代码—拆分、合并与矩阵加法

```

1 Matrix combieMatrix(Matrix C1,Matrix C2,Matrix C3,Matrix C4,int halfsize)
2 {
3     Matrix C=createMatrix(halfsize*2);
4     for(int i=0;i<halfsize*2;i++)
5     {
6         for(int j=0;j<halfsize*2;j++)
7         {
8             if(i<halfsize && j<halfsize)
9             {
10                C[i][j]=C1[i][j];
11            }
12            else if(i<halfsize && j>=halfsize)
13            {
14                C[i][j]=C2[i][j-halfsize];
15            }
16            else if(i>=halfsize && j<halfsize)
17            {
18                C[i][j]=C3[i-halfsize][j];
19            }
20            else
21            {
22                C[i][j]=C4[i-halfsize][j-halfsize];
23            }
24        }
25    }
26    return C;
27 }
28
29 SplitMat SplitMatrix(Matrix mat,int size)
30 {
31     int halfsize=size/2;
32     Matrix m11,m12,m21,m22;
33     SplitMat splitmat;
34     m11=createMatrix(halfsize);
35     m12=createMatrix(halfsize);
36     m21=createMatrix(halfsize);
37     m22=createMatrix(halfsize);
38     for(int i=0;i<halfsize;i++)
39     {
40         for(int j=0;j<halfsize;j++)

```

```

41     {
42         m11[i][j]=mat[i][j];
43         m12[i][j]=mat[i][j+halfsize];
44         m21[i][j]=mat[i+halfsize][j];
45         m22[i][j]=mat[i+halfsize][j+halfsize];
46     }
47 }
48 splitmat.m11=m11;
49 splitmat.m12=m12;
50 splitmat.m21=m21;
51 splitmat.m22=m22;
52 return splitmat;
53 }
54
55 Matrix AddMatrix(Matrix A,Matrix B,int size)
56 {
57     Matrix C=createMatrix(size);
58     for(int i=0;i<size;i++)
59     {
60         for(int j=0;j<size;j++)
61         {
62             C[i][j]=A[i][j]+B[i][j];
63         }
64     }
65     return C;
66 }

```

Listing 5.15: algo/src/matrix/1.c —combieMatrix / SplitMatrix / AddMatrix

这里 `SplitMatrix` 实际上是在**复制**数据生成四块新矩阵（而不是简单地划分子数组视图）。虽然代价是额外的 $O(n^2)$ 内存，但教学意义很强：能清楚看到「大矩阵 = 四块小矩阵合并」的结构。

5.6.5 C 代码—递归乘法主体

```

1 Matrix SquareMatrixMultiply(Matrix A,Matrix B,int size)
2 {
3     Matrix C=createMatrix(size);
4     if(size==1)
5     {
6         C[0][0]=A[0][0]*B[0][0];
7     }
8     else
9     {
10        int halfsize=size/2;
11        Matrix C1=createMatrix(halfsize);
12        Matrix C2=createMatrix(halfsize);

```

```

13     Matrix C3=createMatrix(halfsize);
14     Matrix C4=createMatrix(halfsize);
15
16     SplitMat splitA=SplitMatrix(A,size);
17     SplitMat splitB=SplitMatrix(B,size);
18
19     Matrix D1=createMatrix(halfsize);
20     Matrix D2=createMatrix(halfsize);
21     Matrix D3=createMatrix(halfsize);
22     Matrix D4=createMatrix(halfsize);
23     Matrix D5=createMatrix(halfsize);
24     Matrix D6=createMatrix(halfsize);
25     Matrix D7=createMatrix(halfsize);
26     Matrix D8=createMatrix(halfsize);
27
28     D1=SquareMatrixMultiply(splitA.m11,splitB.m11,halfsize);
29     D2=SquareMatrixMultiply(splitA.m12,splitB.m21,halfsize);
30     D3=SquareMatrixMultiply(splitA.m11,splitB.m12,halfsize);
31     D4=SquareMatrixMultiply(splitA.m12,splitB.m22,halfsize);
32     D5=SquareMatrixMultiply(splitA.m21,splitB.m11,halfsize);
33     D6=SquareMatrixMultiply(splitA.m22,splitB.m21,halfsize);
34     D7=SquareMatrixMultiply(splitA.m21,splitB.m12,halfsize);
35     D8=SquareMatrixMultiply(splitA.m22,splitB.m22,halfsize);
36
37     C1=AddMatrix(D1,D2,halfsize);
38     C2=AddMatrix(D3,D4,halfsize);
39     C3=AddMatrix(D5,D6,halfsize);
40     C4=AddMatrix(D7,D8,halfsize);
41     C=combieMatrix(C1,C2,C3,C4,halfsize);
42
43     freeMatrix(C1,halfsize);
44     freeMatrix(C2,halfsize);
45     freeMatrix(C3,halfsize);
46     freeMatrix(C4,halfsize);
47     freeMatrix(D1,halfsize);
48     freeMatrix(D2,halfsize);
49     freeMatrix(D3,halfsize);
50     freeMatrix(D4,halfsize);
51     freeMatrix(D5,halfsize);
52     freeMatrix(D6,halfsize);
53     freeMatrix(D7,halfsize);
54     freeMatrix(D8,halfsize);
55 }
56 return C;
57 }

```

Listing 5.16: algo/src/matrix/1.c —SquareMatrixMultiply（分治法矩阵乘法）

递归基例：当 `size == 1` 时， $C[0][0] = A[0][0] \times B[0][0]$ ，只是一个标量乘法。

对于更大的矩阵：

1. 把 A、B 各切成四块
2. 递归计算 8 次子乘积 $\{D1..D8\}$
3. 把子和合并回结果矩阵 C

这个版本仍然是 $O(n^3)$ （实际上是 8 次子乘法 + 4 次加法），只是把循环换成了递归。如果想真正优化复杂度，可以用 **Strassen 算法**——把 8 次子乘法减少为 7 次，代价是增加更多加法。Strassen 的复杂度为 $O(n^{\log_2 7}) \approx O(n^{2.807})$ 。

5.6.6 C 代码—主程序验证

```

1 int main()
2 {
3     int n=3;
4     Matrix A=createMatrix(n);
5     Matrix B=createMatrix(n);
6     int count=1;
7     for(int i=0;i<n;i++)
8     {
9         for(int j=0;j<n;j++)
10        {
11            A[i][j]=count;
12            B[i][j]=count++;
13        }
14    }
15    printMatrix(A,n);
16
17    Matrix padA=padMatrix(A,n);
18    printMatrix(padA,nextpoweroftwo(n));
19    Matrix padB=padMatrix(B,n);
20    Matrix C=SquareMatrixMultiply(padA,padB,nextpoweroftwo(n));
21    printMatrix(C,n);
22
23    freeMatrix(C,n);
24    freeMatrix(padA,nextpoweroftwo(n));
25    freeMatrix(padB,nextpoweroftwo(n));
26    freeMatrix(A,n);
27    freeMatrix(B,n);
28    return 0;
29 }
```

Listing 5.17: algo/src/matrix/1.c —main

测试矩阵 3×3 ，因为 3 不是 2 的幂，会先被补零到 4×4 ，然后递归相乘，最后输出时只取左上角的 3×3 。

编译运行：

```
1 clang -o matrix algo/src/matrix/1.c -lm
2 ./matrix
```

注意需要 `-lm` 链接数学库，因为用到了 `pow/log2/ceil`。

5.6.7 从底层视角看矩阵乘法

和前面的排序算法相比，矩阵乘法让你直接接触到两件事：

- **二维数组的内存布局**：C 里 `int A[3][3]` 是行优先的，等价于 9 个连续 `int`。这与第 2 章汇编里处理字符串数组的做法是同一条路——先算偏移，再访问内存。
- **访存模式与性能**：朴素的三重循环对 A 逐行、对 B 逐列访问。在大矩阵上，B 的列访问会严重打击缓存命中率。把 B 先转置、或者改写循环顺序（如 `ikj` 形式），可以让实际运行时间快几倍。这也是下一步如果要做性能优化的方向。

5.7 从 C 代码到汇编—底层视角

写 C 代码对理解汇编编程有直接帮助，反过来也是一样。这一节做三个连接。

5.7.1 函数调用 vs 栈操作

C 里写一个递归函数（比如 `Mergesort`），编译器背后做的事是：

```
1 bl Mergesort           // 保存返回地址到 x30，然后跳转
2 ...                   // 递归展开时，每次都压栈保存 x29/x30
```

理解了 C 函数的**调用约定**（calling convention），汇编里写 `stp x29, x30, [sp, #-16]!` 的保存动作就很自然——每一次 `bl` 都会覆盖 `x30`，所以递归前必须把它存进栈。

5.7.2 数组与指针 vs [base, index, shift]

C 里写 `A[i] = B[j]`，其实就是：

```
1 // A[i] 对应 基地址 + i * sizeof(元素)
2 // 汇编里就是 [x0, x1, lsl #2] （元素 4 字节）
3 // 这就是指针算术的本质
```

理解这一点后，再看第 2 章里写的 `string_ptrs[i]` 和 `string_lens[i]`，就不会觉得是魔法了——只是把指针当作索引值，加上偏移。

5.7.3 结构体与内存对齐

矩阵乘法里定义了 `SplitMat` 这样的结构体，它包含四个 `Matrix` 类型的子矩阵指针。每个结构体成员在内存中的偏移取决于它的大小和编译器对对齐的要求。这与 Verilog 芯片里寄存器地址排布的思路是一致的——一切数据最终都落到「地址 + 大小」这两件事上。

5.7.4 从算法到硬件的路径

现在你在项目中走过了一条完整的链路：

层面	你写的代码	关注点
C 算法	<code>src/sort/*.c</code> 等	时间复杂度、分治
用户态汇编	<code>ISA/hello.s</code>	寄存器、栈、系统调用
裸机汇编	<code>QEMU/start.s</code>	无 OS、UART 串口
Verilog	<code>Chip/src/*.v</code>	时钟、状态机、寄存器

从算法出发，你可以反过来问：「这个 $O(n^2)$ 的循环如果变成硬件电路，它的流水线会是什么样？」「这一次内存访问要走几个时钟周期？」——这类问题让算法和硬件设计自然地连通。

下一步可以做的事

- 用 LLDB 观察 `Mergesort` 递归调用时栈的变化
- 把 `heapsort.c` 实现出来（当前文件还是空的）
- 写一个最小的 C 标准库子集（`string.h` / `stdlib.h`）
- 为矩阵乘法加入性能计时，对比朴素三重循环、ikj 循环、分治法的实际速度